



UNIVERSIDADE
FEDERAL DO CEARÁ

Relatório Final

Manutenção De Software - 01 - 2026.1

Profa. Carla Ilane Moreira Bezerra

Autores

Edivar Cruz Carvalho Filho - 565857

Julian César Pereira Cardoso - 553782

Sumário

Sumário	1
1. Caracterização do Projeto	3
2. Métricas Antes da Refatoração	3
2.1 Radon Antes da Refatoração	3
2.1.1 Complexidade Ciclométrica por Arquivo Antes da Refatoração	3
2.1.2 Complexidade Ciclométrica por Função Antes da Refatoração	4
2.1.3 Índice de Manutenibilidade Antes da Refatoração	4
2.1.4 Métricas brutas Antes da Refatoração	5
2.2 CodeCarbon Antes da Refatoração	5
2.3 Pylint Antes da Refatoração	5
2.3.1 Distribuição do Smells Antes da Refatoração	5
2.3.2 Distribuição dos Problemas de Código Antes da Refatoração	6
2.3.3 Arquivos mais Críticos Antes da Refatoração	6
2.3.4 Score Projeto Antes da Refatoração	7
2.4 Pytest Antes da Refatoração	7
2.4.1 Cobertura de testes Antes da Refatoração	7
2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração	9
3. Processo de Refatoração	9
3.1 too-many-statements	9
3.1.1 Ocorrência Número 1	9
3.1.2 Ocorrência Número 2	10
3.2 too-many-instance-attributes	11
3.3 too-many-branches	11
4. Métricas Após a Refatoração	11
4.1 Radon Após a Refatoração	11
4.1.1 Complexidade Ciclométrica por Arquivo Após a Refatoração	11
4.1.2 Complexidade Ciclométrica por Função Após a Refatoração	11
4.1.3 Índice de Manutenibilidade Após a Refatoração	12
4.1.4 Métricas Brutas Após a Refatoração	12
4.2 CodeCarbon Após a Refatoração	12
4.3 Pylint Após a Refatoração	13
4.3.1 Distribuição do Smells Após a Refatoração	13
4.3.2 Distribuição dos Problemas de Código Após a Refatoração	13
4.3.3 Arquivos mais Críticos Após a Refatoração	13
4.3.4 Score Projeto Após a Refatoração	13
4.4 Pytest Após a Refatoração	13
4.4.1 Cobertura de testes Após a Refatoração	13
4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração	13
5. Comparação dos Resultados	14

6. Percepções sobre a Prática	14
7. Links	14

1. Caracterização do Projeto

Nome Projeto: Supervision

O projeto escolhido foi o **Supervision**, uma biblioteca open source desenvolvida em Python voltada para aplicações de visão computacional. Seu principal objetivo é facilitar o processamento, manipulação, visualização e análise de resultados produzidos por modelos de inteligência artificial aplicados a imagens e vídeos.

A biblioteca funciona como uma camada de apoio para projetos que utilizam detecção de objetos, segmentação, rastreamento e outras tarefas relacionadas à visão computacional. Entre suas funcionalidades estão a criação de anotações visuais em imagens, manipulação de coordenadas e regiões, acompanhamento de objetos entre quadros de vídeo e geração de métricas para avaliação de modelos.

O Supervision foi desenvolvido para simplificar tarefas que normalmente exigiriam grande quantidade de código manual, oferecendo abstrações reutilizáveis e componentes prontos para acelerar o desenvolvimento de aplicações de visão computacional.

Por ser um projeto open source ativo e relativamente amplo, ele reúne diferentes módulos e responsabilidades dentro da mesma base de código, o que o torna um bom exemplo para observar práticas reais de organização, evolução e manutenção de software em projetos utilizados pela comunidade.

2. Métricas Antes da Refatoração

Nesta seção você vai apresentar as principais métricas obtidas antes da refatoração para cada ferramenta, não precisa ser todas as métricas. Confira um exemplo abaixo:

2.1 Radon Antes da Refatoração

2.1.1 Complexidade Ciclômica por Arquivo Antes da Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela antes da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	funções	cc_mediana	cc_max	cc_soma	pior_rank	pior_funcao
supervision/detection/utils/internal.py	8	10.12	22	81	D	process_robotflow_result
supervision/detection/vim.py	15	8.67	33	130	E	from_google_gemini_2_0
supervision/detection/utils/poligon.py	2	8.00	9	16	B	filter_polygons_by_area
supervision/metrics/mean_average_precision.py	32	7.03	29	225	D	_accumulate
supervision/tracker/byte_tracker/core.py	7	6.86	26	48	D	update_with_tensors

Tabela de melhores métricas

arquivo	funções	cc_mediana	cc_max	cc_soma	pior_rank	pior_funcao
supervision/annotators/base.py	1	1.0	1	1	A	annotate
supervision/geometry/core.py	11	1.0	1	11	A	list
supervision/metrics/core.py	3	1.0	1	3	A	update
supervision/tracker/by_tracker/utils.py	4	1.25	2	5	A	init
supervision/tracker/by_tracker/kalman_filter.py	6	1.33	2	8	A	init

2.1.2 Complexidade Ciclômática por Função Antes da Refatoração

Apresente as funções do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela antes da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
src/supervision/key_points/core.py	method	KeyPoints.getitem	E	39
src/supervision/detection/vlm.py	function	from_google_gemini_2_5	E	33
src/supervision/metrics/mean_average_precision.py	method	COCOEvaluator_accumulae	D	29
src/supervision/metrics/mean_average_precision.py	method	COCOEvaluator_evaluate_image	D	28
src/supervision/tracker/byte_tracker/core.py	method	ByteTrack.update_with_tensors	D	26

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
src/supervision/annotators/base.py	method	BaseAnnotator.annotate	A	1
src/supervision/annotators/core.py	method	_normalize_color_input	A	1
src/supervision/annotators/core.py	function	_BaseLabelAnnotator.init	A	1
src/supervision/annotators/core.py	function	BoxAnnotator.init	A	1
src/supervision/annotators/core.py	function	OrientedBoxAnnotator.init	A	1

2.1.3 Índice de Manutenibilidade Antes da Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela antes da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	mi	rank_mi
src/supervision/annotators/core.py	0.5164	C
src/supervision/metrics/mean_average_precision.py	11.3722	C
src/supervision/detection/core.py	15.2378	B
src/supervision/detection/compact_mask.py	25.2684	B
src/supervision/detection/utils/converters.py	26.8962	A

Tabela de melhores métricas

arquivo	mi	rank_mi
src/supervision/config.py	100.0	A
src/supervision/annotators/base.py	100.0	A
src/supervision/annotators/init.py	100.0	A
src/supervision/assets/list.py	100.0	A
src/supervision/assets/init.py	100.0	A

2.1.4 Métricas brutas Antes da Refatoração

Essas métricas podem ser encontradas no arquivo *raw_por_arquivo_e_total_antes.csv*. Traga apenas o total, que se encontra no final do csv.

loc total	lloc total	sloc total	comments total	multi
33953	10762	19554	522	9129

2.2 CodeCarbon Antes da Refatoração

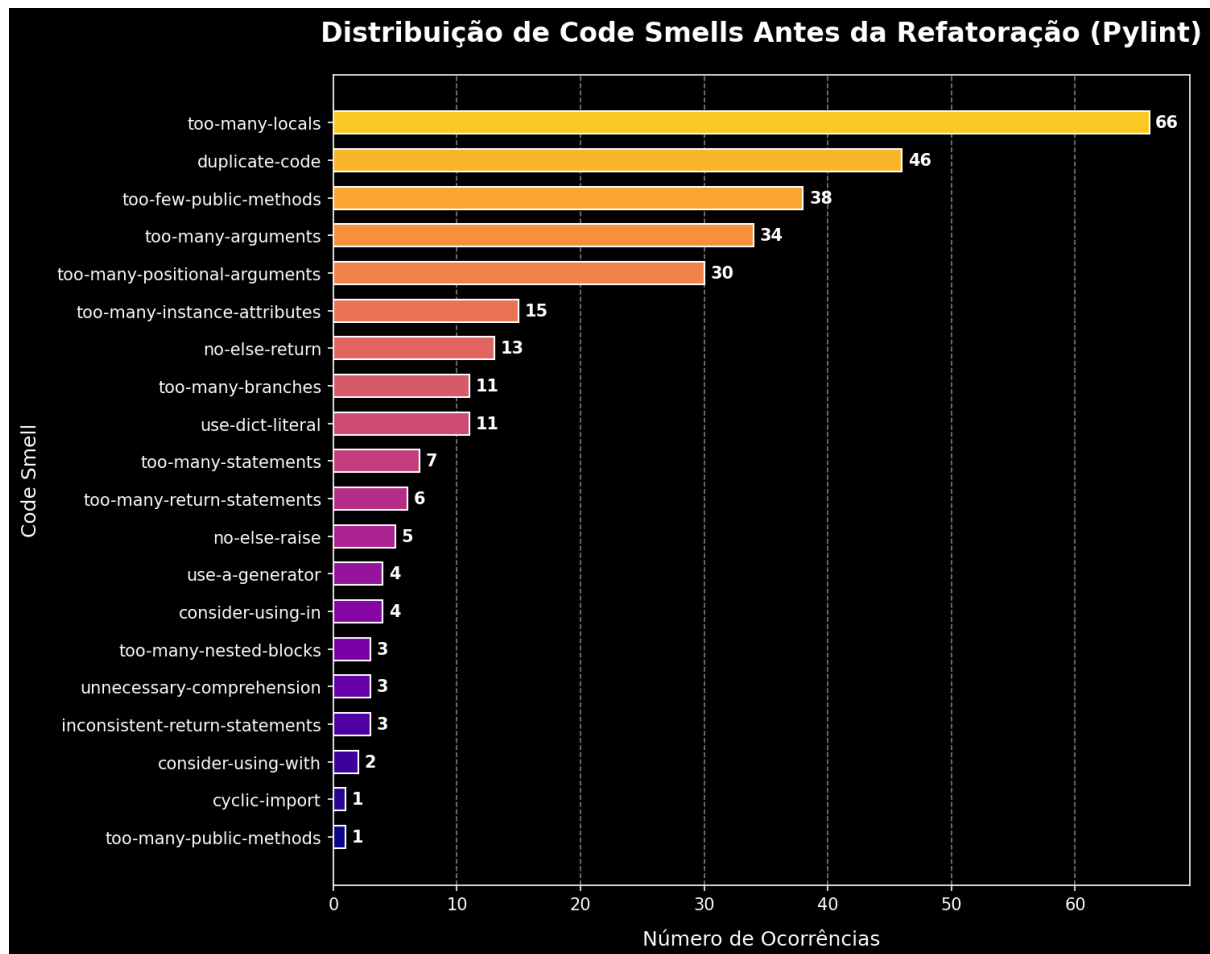
Preencha a tabela abaixo com as métricas ambientais antes da refatoração.

duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
88.1082	1.9513e-05	2.2147e-07	7.0576e-05	1.2783e-04	1.9841e-04

2.3 Pylint Antes da Refatoração

2.3.1 Distribuição do Smells Antes da Refatoração

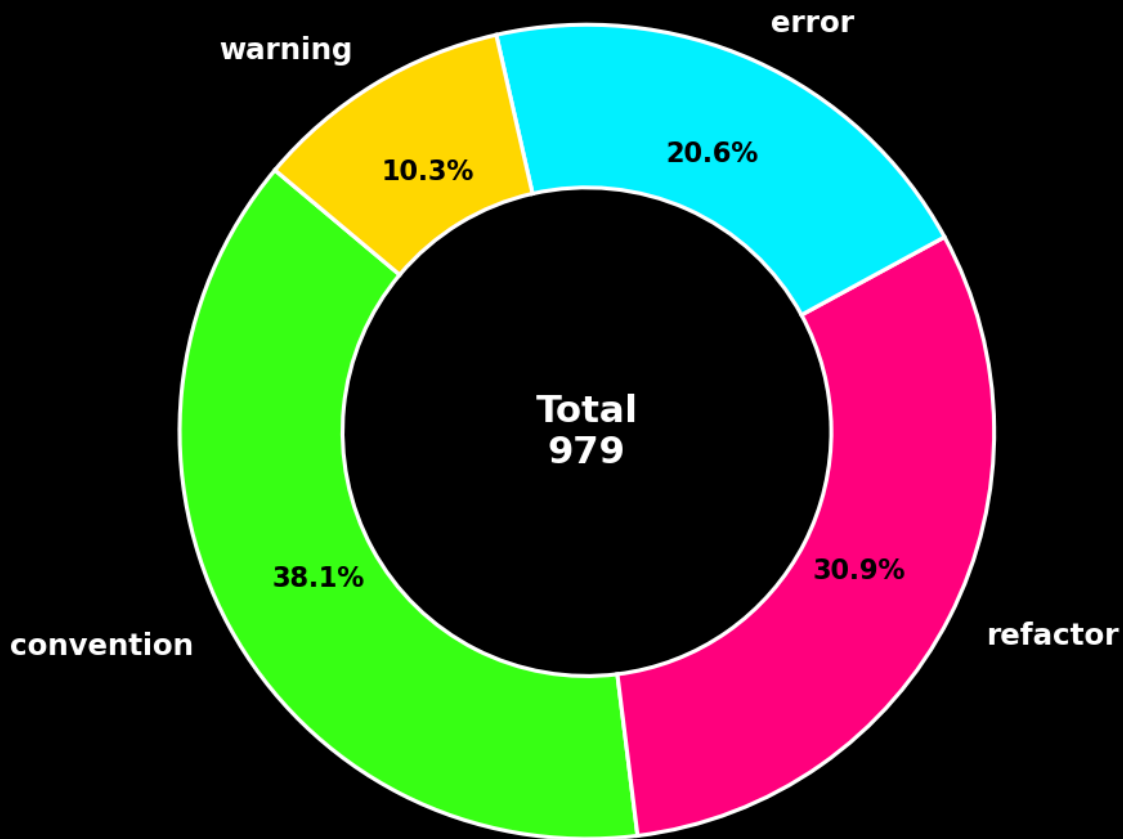
A partir do JSON `pylint_ranking_smells_antes.json` gere um gráfico para representar a distribuição dos code smells antes da refatoração. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.



2.3.2 Distribuição dos Problemas de Código Antes da Refatoração

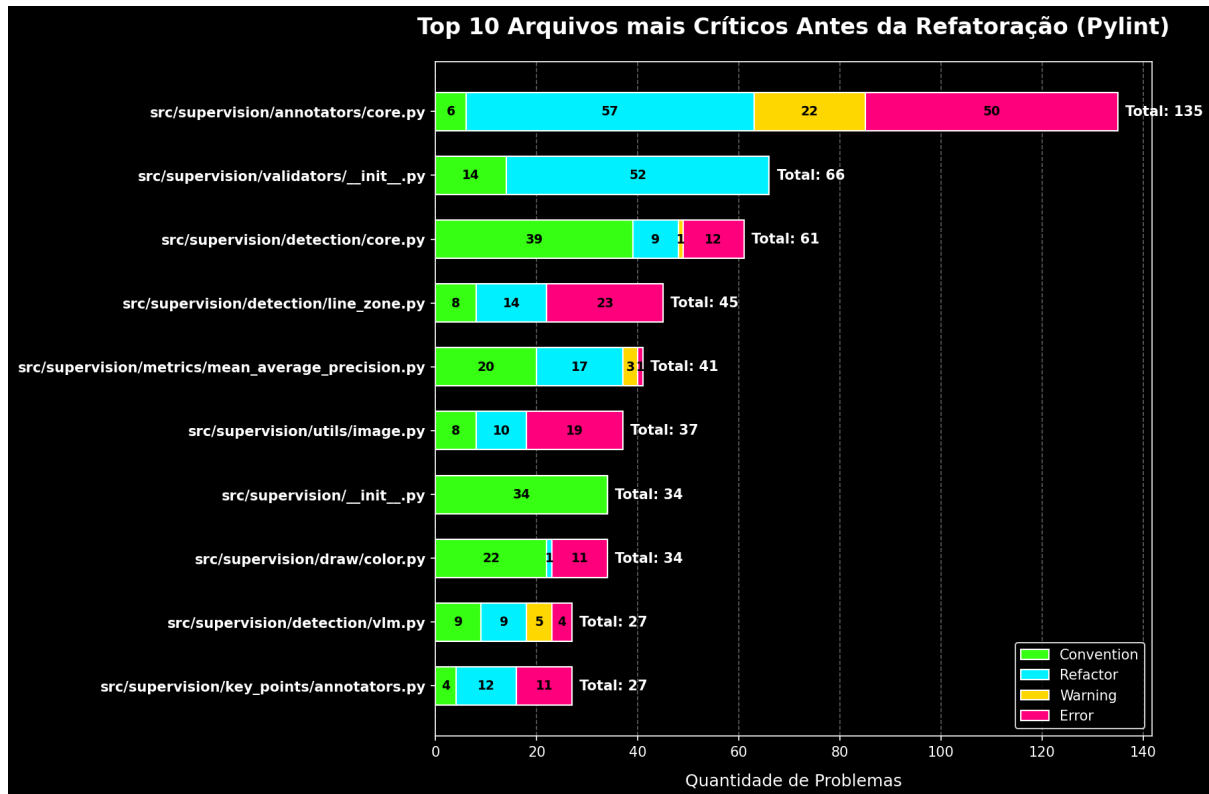
A partir do JSON `pylint_distribuicao_categorias_antes.json` gere um gráfico para representar a distribuição entre as 4 categorias de problemas de código antes da refatoração. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.

Distribuição de Categorias Pylint Antes da Refatoração



2.3.3 Arquivos mais Críticos Antes da Refatoração

A partir do JSON `pylint_arquivos_criticos_antes.json` gere um gráfico para representar a distribuição dos 10 arquivos mais críticos antes da refatoração. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.



2.3.4 Score Projeto Antes da Refatoração

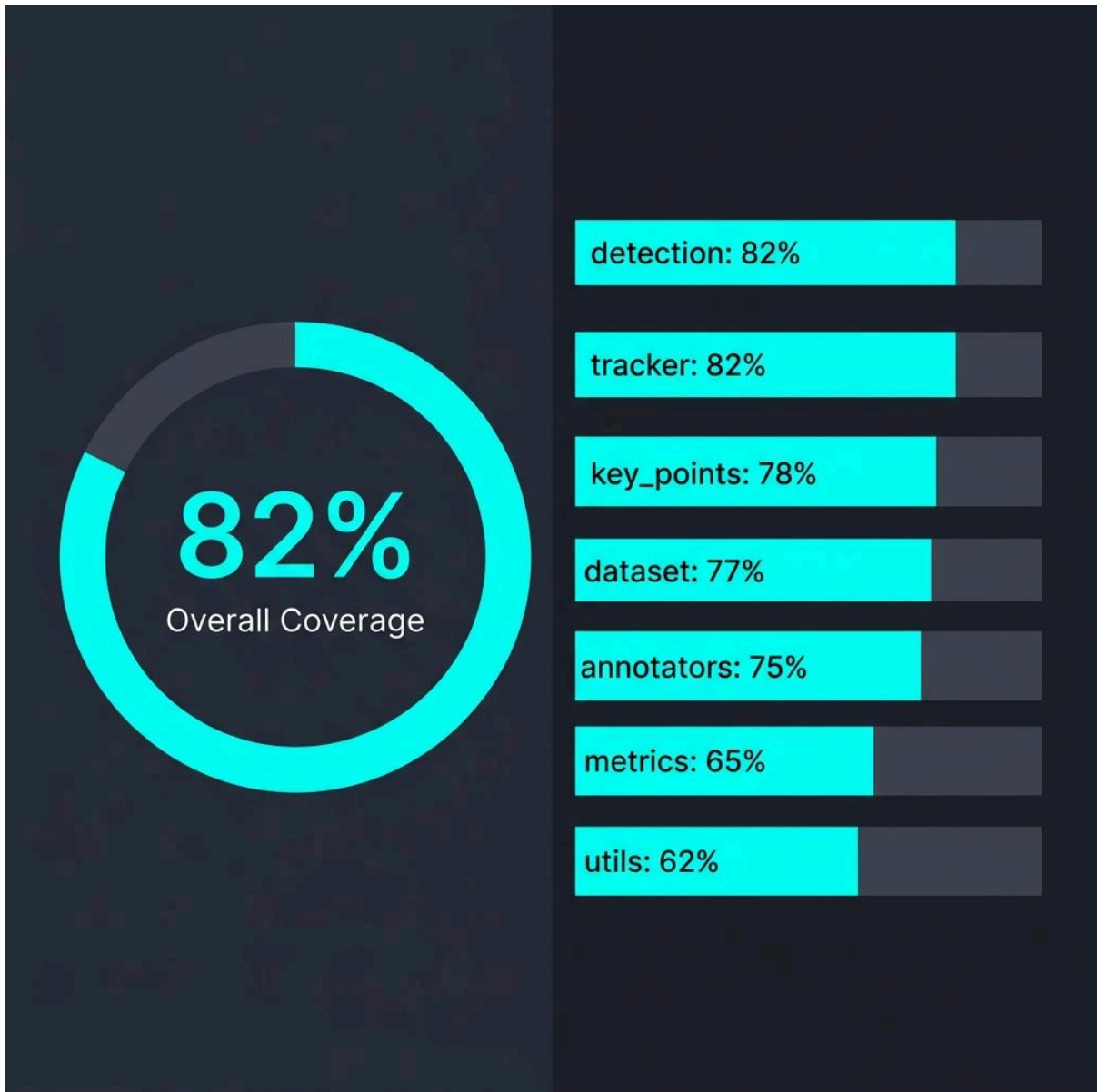
A partir do text `pylint_score_antes.txt` informe o score que o pylint deu ao projeto antes da refatoração. Não precisa ser uma imagem, fica a critério de vocês.

7.98/10

2.4 Pytest Antes da Refatoração

2.4.1 Cobertura de testes Antes da Refatoração

Gere um gráfico da cobertura de testes do projeto antes da refatoração. Para isso, basta visualizar a cobertura de testes na página HTML, copiar os dados da tabela e gerar o gráfico com o auxílio de uma LLM. Para visualizar a página de cobertura de testes antes da refatoração, execute o seguinte comando: `~`. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.



[arquivo_coverage](#)

2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração

Aqui você deve informar quantos testes falharam e quantos testes passaram antes da refatoração. Para isso, basta visualizar a página HTML `pytest_antes.html`. Para acessá-la, execute o seguinte comando: `start "metrics-before-pytest\pytest_antes.html"`.

Quantidade de testes que passaram: 2064

Quantidade de testes que falharam: 4

3. Processo de Refatoração

Para o processo de refatoração foram utilizados dois modelos de linguagem. As primeiras ocorrências foram tratadas com o modelo **DeepSeek**, enquanto parte das demais refatorações foi realizada com o **GPT-5.4 mini (plano gratuito)**. A escolha dos modelos permitiu comparar diferentes estratégias de análise e geração de código durante o processo de remoção dos code smells.

3.1 too-many-statements

Primeiramente, informe a quantidade de ocorrências desse tipo de smell.

Quantidade de ocorrências: 7

3.1.1 Ocorrência Número 1

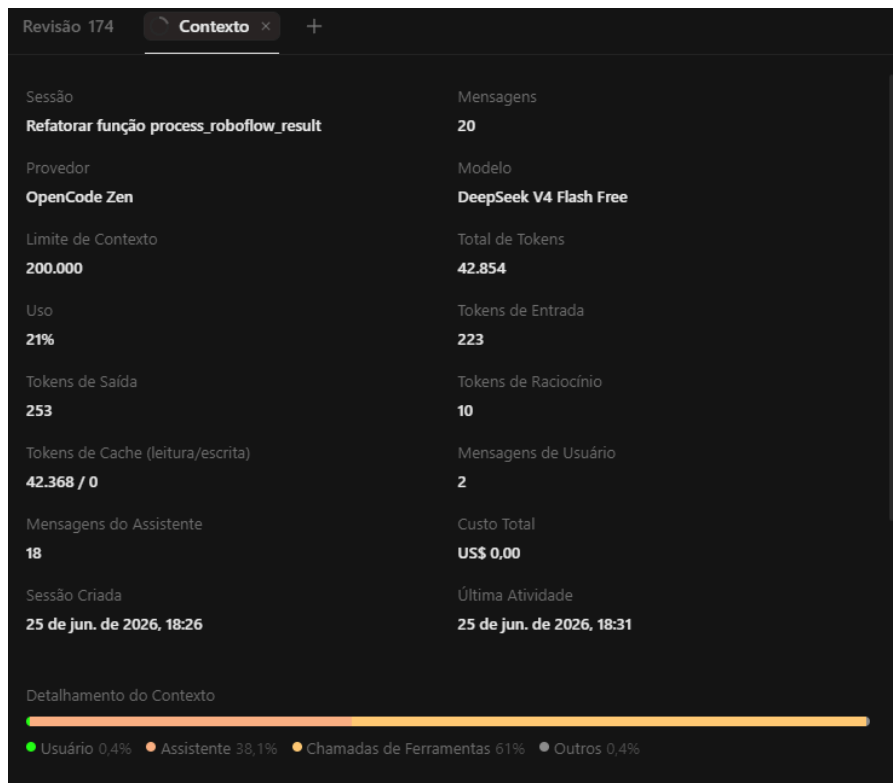
Tabela de identificação

module	obj	message
<code>supervision.detection.utils.internal</code>	<code>process_roboflow_result</code>	Too many statements (62/50)

Seguiu o plano de refatoração: Sim. O plano de refatoração proposto foi seguido integralmente, pois apresentou uma solução compatível com a recomendação do Pylint para esse tipo de code smell. A estratégia reduziu a quantidade de instruções da função sem alterar seu comportamento, preservando a compatibilidade com o restante do projeto e sem introduzir novos problemas relevantes de qualidade.

Link da publicação OpenCode: <https://opnecd.ai/share/dfiedAw3>

Print consumo de token:



Análise da refatoração: A refatoração foi concluída rapidamente e eliminou a ocorrência do smell com poucas alterações estruturais. A LLM optou por extrair partes da lógica em métodos auxiliares, reduzindo a quantidade de instruções da função principal e tornando o código mais organizado e legível.

3.1.2 Ocorrência Número 2

Tabela de identificação

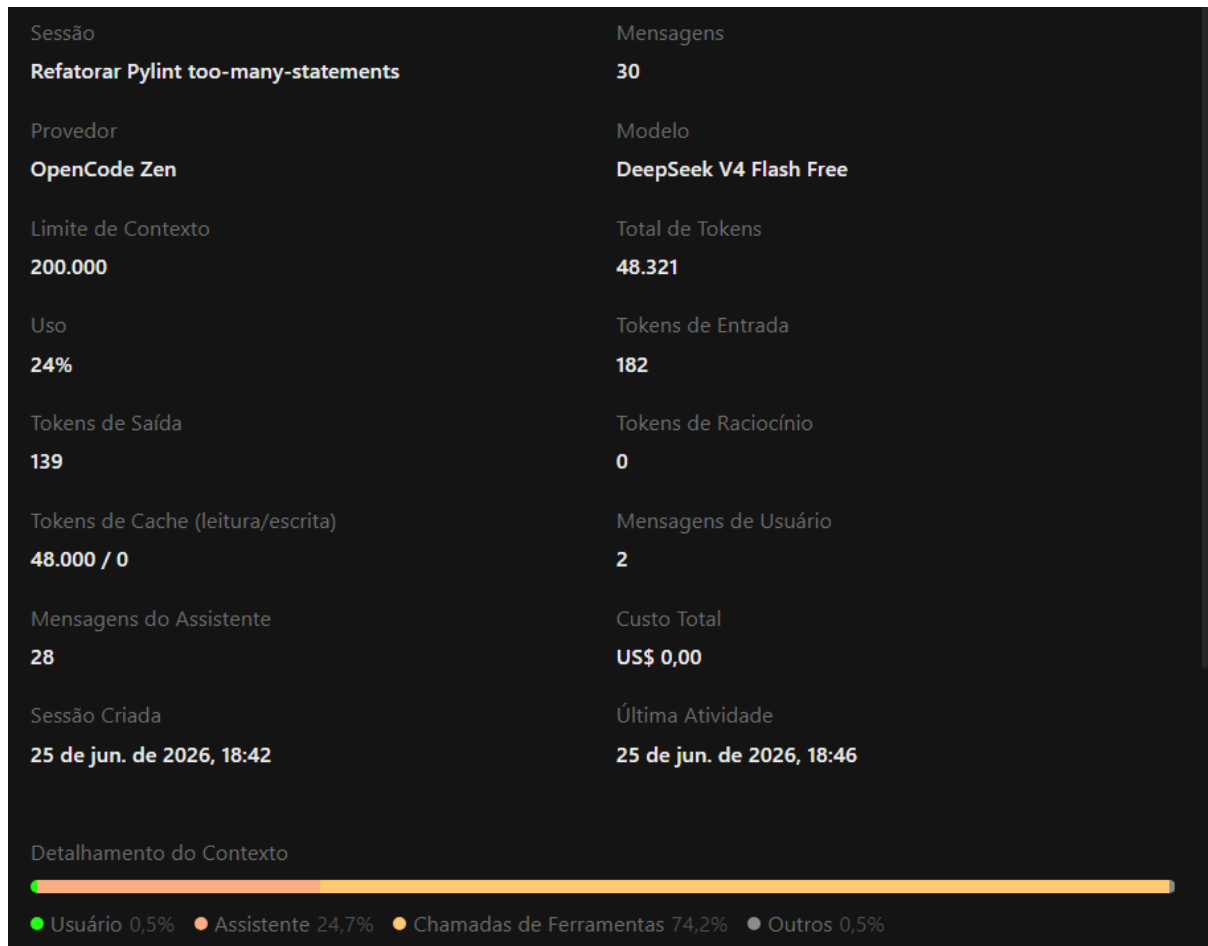
module	obj	message
supervision.key_point s.core	KeyPoints.__getitem__	Too many statements (53/50)

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

Seguiu o plano de refatoração: Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Link da publicação OpenCode: <https://opncd.ai/share/faMclg2q>

Print consumo de token:



Análise da refatoração: A estratégia proposta foi semelhante à anterior, concentrando-se na decomposição da função em trechos menores. O processo ocorreu de forma rápida, removeu o smell identificado pelo Pylint e preservou o comportamento esperado da implementação.

3.1.3 Ocorrência Número 3

Tabela de identificação

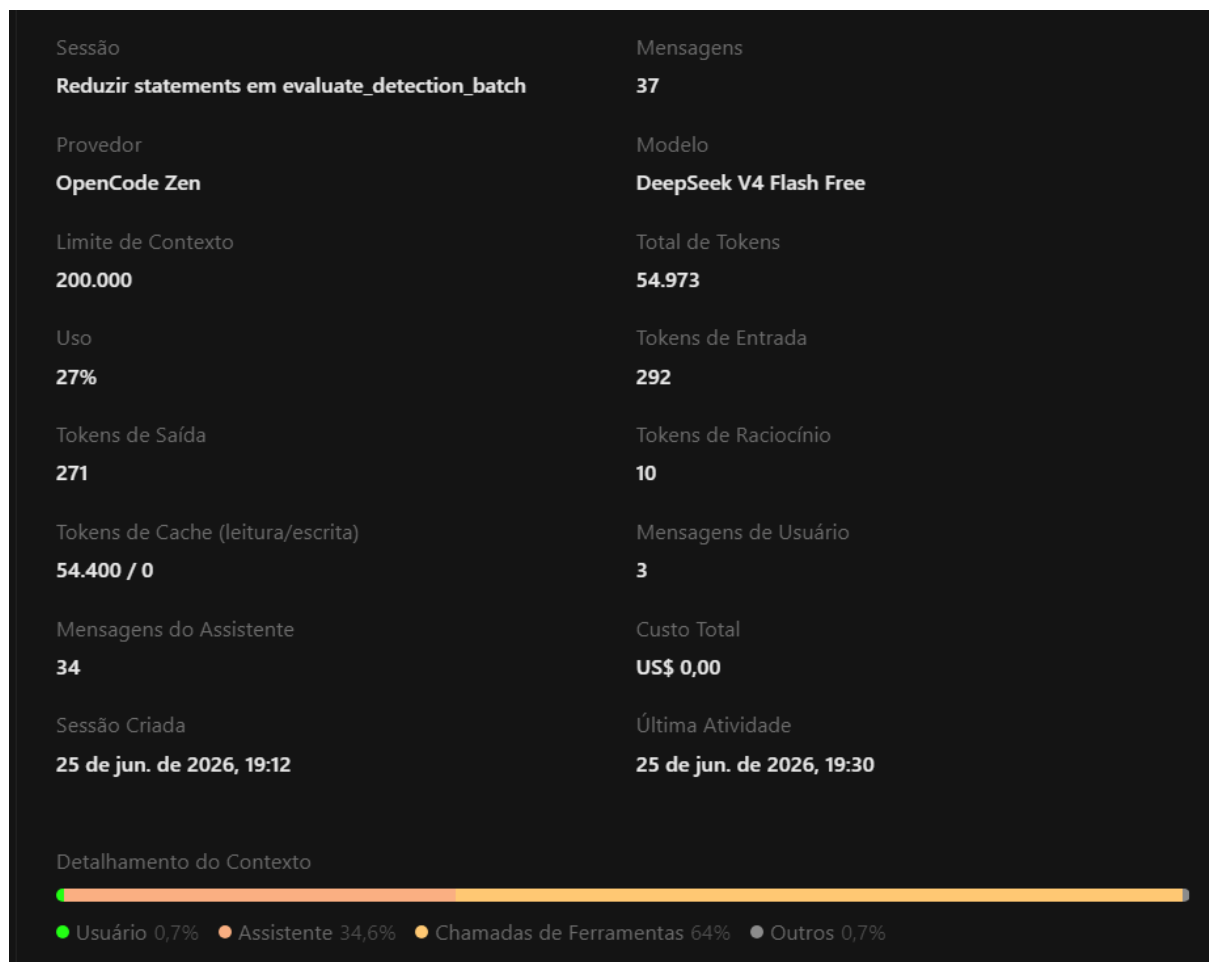
module	obj	message
supervision.metrics.detection	ConfusionMatrix.evaluate_detection_batch	Too many statements (51/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Também deve informar o link da publicação gerada no OpenCode e adicionar um print do consumo de tokens referente à refatoração dessa ocorrência.

Link da publicação OpenCode: <https://opncd.ai/share/JJnxJrz3>

Print consumo de token:



Análise da refatoração: A refatoração eliminou o code smell sem alterar a funcionalidade da aplicação. Entretanto, observou-se que o DeepSeek executou toda a suíte de testes durante o processo, aumentando significativamente o tempo necessário para concluir a tarefa. Apesar desse custo adicional, o resultado final foi satisfatório.

3.1.4 Ocorrência Número 4

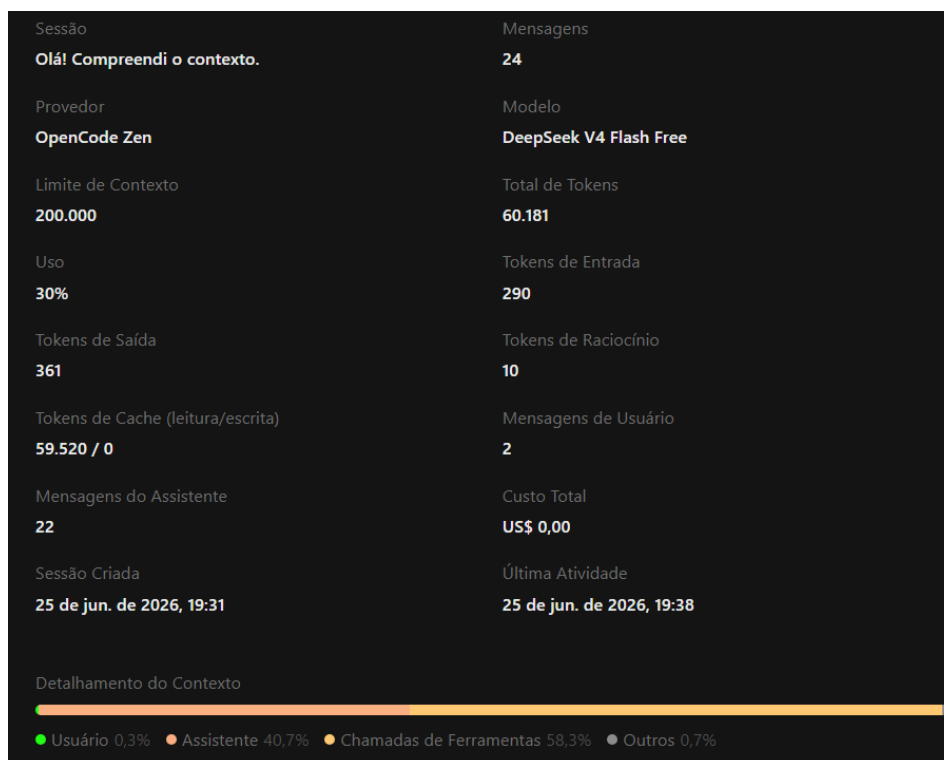
Tabela de identificação

module	obj	message
supervision.metrics.mean_average_precision	COCOEvaluator._accumulate	Too many statements (77/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Link da publicação OpenCode: <https://opncd.ai/share/Wr7UpMgi>

Print consumo de token:



Análise da refatoração: A refatoração foi simples e objetiva, seguindo o plano inicialmente proposto pela LLM. O smell foi removido por meio da reorganização da lógica interna da função, melhorando sua legibilidade sem comprometer seu funcionamento.

3.1.5 Ocorrência Número 5

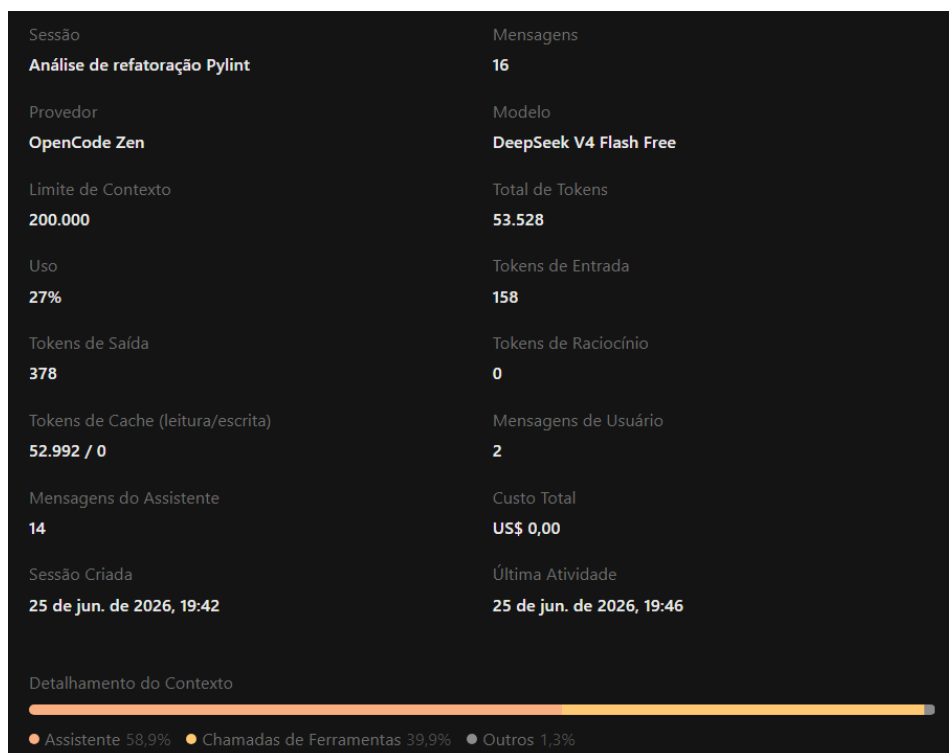
Tabela de identificação

module	obj	message
supervision.tracker.byte_tracker.core	ByteTrack.update_with_tensors	Too many statements (92/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Link da publicação OpenCode: <https://opncd.ai/share/R5tRsWxl>

Print consumo de token:



Análise da refatoração: Apesar da elevada quantidade de instruções presentes na função original, a LLM conseguiu estruturar uma solução consistente utilizando extração de responsabilidades. O resultado foi obtido rapidamente e sem necessidade de alterações adicionais após a validação.

3.1.6 Ocorrência Número 6

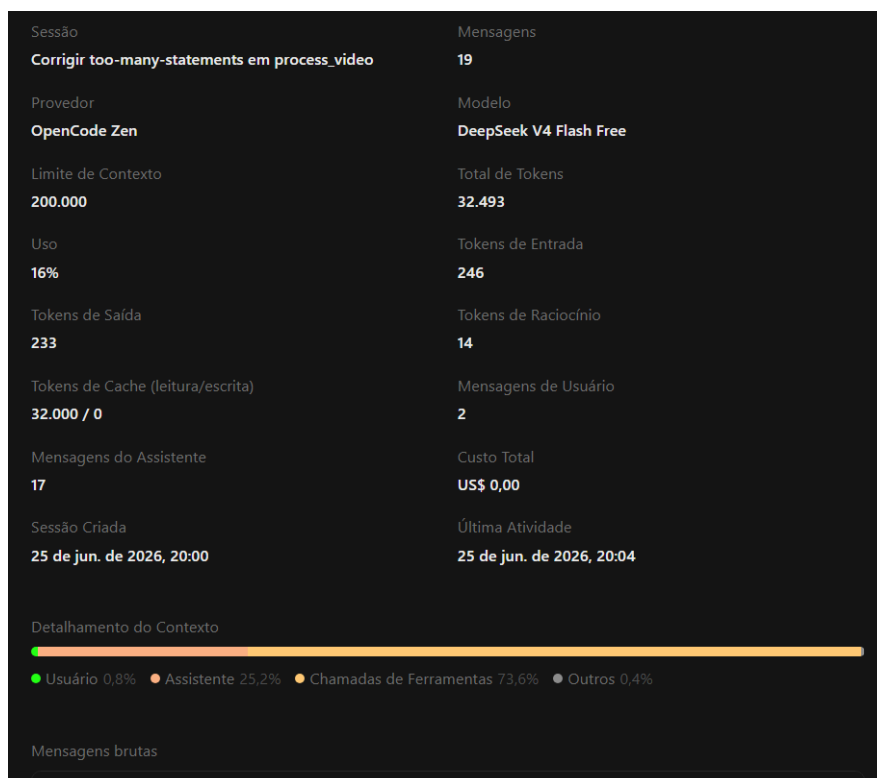
Tabela de identificação

module	obj	message
supervision.utils.video	process_video	Too many statements (56/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Link da publicação OpenCode: <https://opncd.ai/share/hZxksazz>

Print consumo de token:



Análise da refatoração: A refatoração ocorreu de forma eficiente e exigiu poucas modificações no restante do código. A divisão da lógica em métodos menores tornou a implementação mais fácil de compreender e reduziu a complexidade da função principal.

3.1.7 Ocorrência Número 7

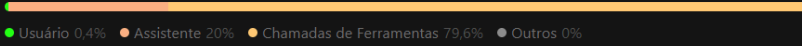
Tabela de identificação

module	obj	message
supervision.detection.vi m"	from_google_gemini_2_5	Too many statements (71/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Link da publicação OpenCode: <https://opncd.ai/share/ycDJxyF1>

Print consumo de token:

Sessão	Mensagens
Too many statements (71/50)	27
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	64.511
Uso	Tokens de Entrada
32%	8.335
Tokens de Saída	Tokens de Raciocínio
223	17
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
55.936 / 0	2
Mensagens do Assistente	Custo Total
25	US\$ 0,00
Sessão Criada	Última Atividade
25 de jun. de 2026, 22:55	25 de jun. de 2026, 23:00
Detalhamento do Contexto	
	

Análise da refatoração: Mesmo tratando uma função relativamente extensa, a LLM conseguiu identificar blocos independentes e reorganizar sua estrutura de maneira consistente. A remoção do smell foi realizada sem impactos observáveis na execução do sistema.

3.2 too-many-instance-attributes

Número de ocorrências: 15

3.2.1 Ocorrência Número 1

Tabela de identificação

Seguiu o plano
de refatoração:

Sim. A proposta

foi seguida por

respeitar a arquitetura existente do projeto. Antes da modificação, a LLM analisou a estrutura da classe e identificou quais atributos poderiam ser reorganizados sem comprometer a compatibilidade da API.

module	obj	message
supervision.annotator s.core	_BaseLabelAnnotator	Too many instance attributes (9/7)

Link da publicação OpenCode: <https://opnecd.ai/share/mTyQ3J6Y>

Print de consumo do token:

Sessão	Mensagens
Pylint R0902 em _BaseLabelAnnotator	24
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	60.230
Uso	Tokens de Entrada
15%	479
Tokens de Saída	Tokens de Raciocínio
195	164
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
59.392 / 0	2
Mensagens do Assistente	Custo Total
22	US\$ 0,00
Sessão Criada	Última Atividade
28 de jun. de 2026, 09:38	28 de jun. de 2026, 09:42
Detalhamento do Contexto	
● Usuário 0,4% ● Assistente 10% ● Chamadas de Ferramentas 89,4% ● Outros 0,2%	

Análise da refatoração: A solução proposta reduziu a quantidade de atributos da classe mantendo sua funcionalidade original. Como as alterações ficaram concentradas em poucos pontos do código, o processo foi concluído rapidamente.

3.2.2 Ocorrência Número 2

Tabela de identificação

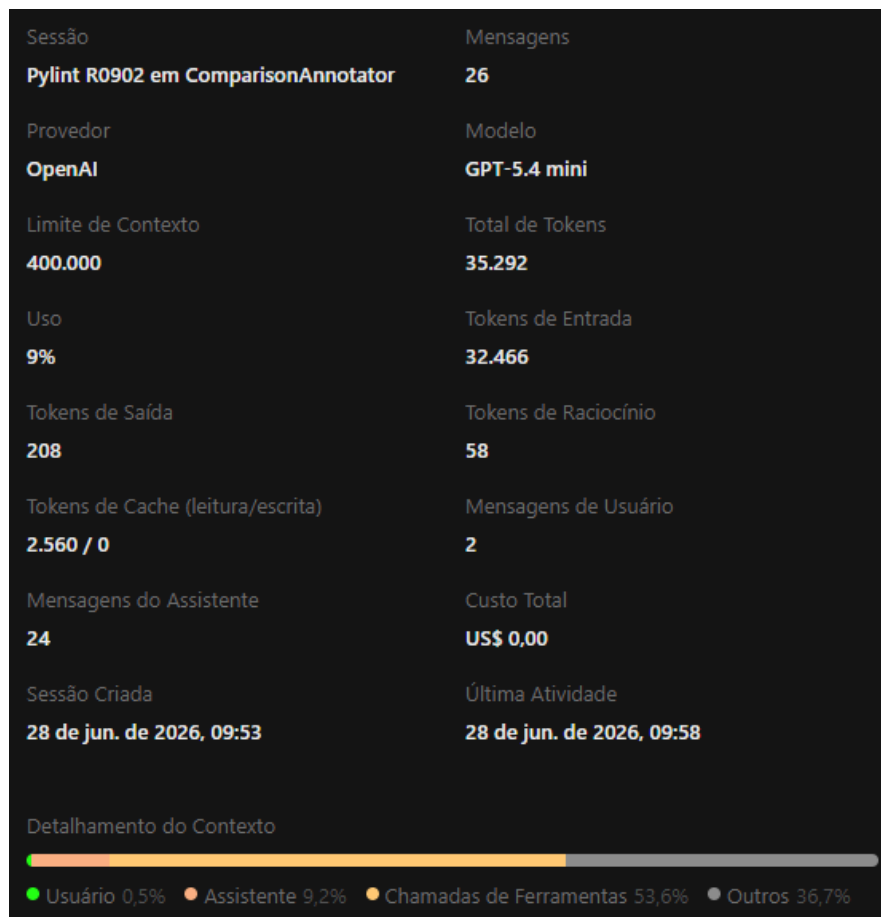
Seguiu o plano de refatoração:

Sim, o plano de refatoração

sugerido resolvia corretamente a ocorrência do smell. Ele verificou a origem do erro, detalhou o caso e propôs uma opção de refatoração que resolvia o problema

Link da publicação OpenCode: <https://opnecd.ai/share/2XoNthsa>

Print de consumo do token:



Análise da refatoração: A LLM realizou uma análise do papel desempenhado pelos atributos da classe antes de propor a refatoração. A solução sugerida atacou diretamente a origem do smell, reorganizando o estado interno sem alterar o comportamento esperado.

3.2.3 Ocorrência Número 3

Tabela de identificação

Seguiu o plano de refatoração:

Sim, o plano de refatoração

sugerido resolvia corretamente a ocorrência do smell. Ele removeu um atributo derivado que era usado minimamente, e começou a usar diretamente o deque.maxlen que já guarda o valor usado

Link da publicação OpenCode: <https://opnecd.ai/share/cLC4noc8>

Print de consumo do token:



Análise da refatoração: A solução consistiu na remoção de um atributo derivado cujo valor já podia ser obtido diretamente por outra estrutura existente. Essa abordagem eliminou redundâncias, reduziu o número de atributos da classe e preservou completamente sua funcionalidade.

3.2.4 Ocorrência Número 4

Tabela de identificação

module	obj	message
--------	-----	---------

Seguiu o plano de refatoração:

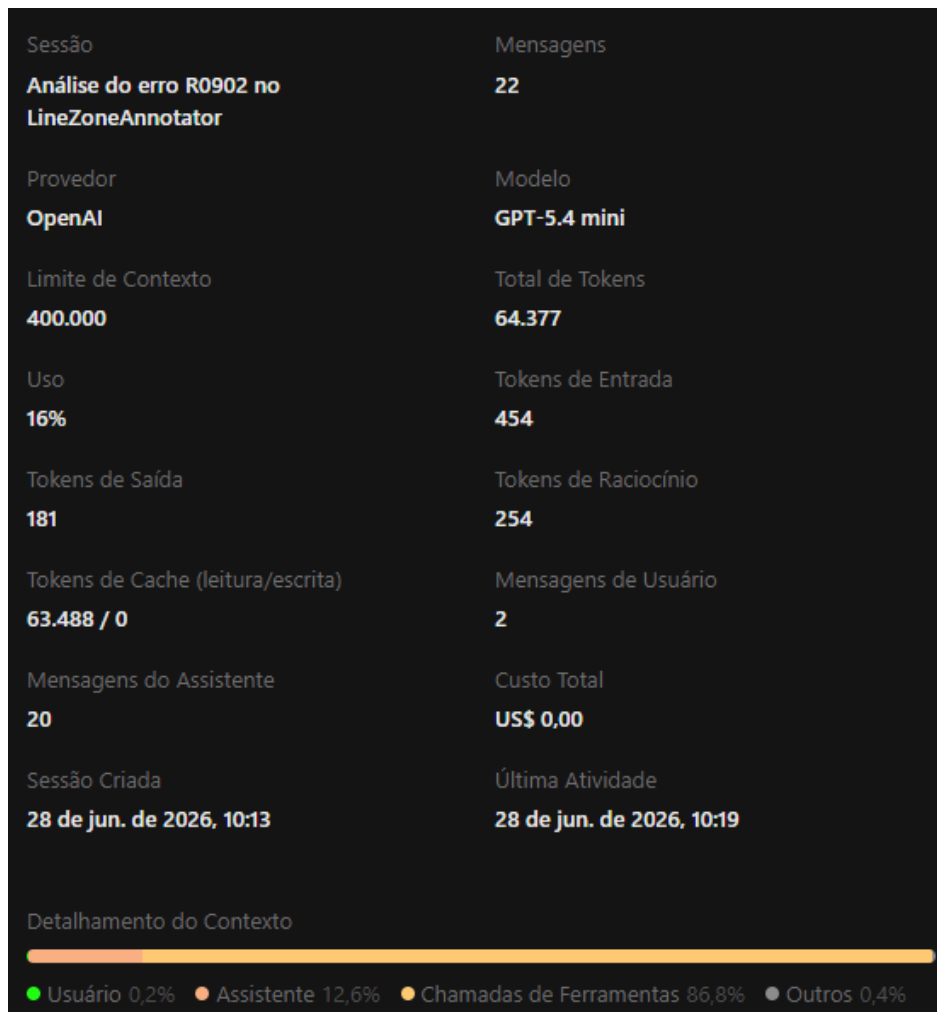
supervision.detection.line_zone	LineZoneAnnotator	Too many instance attributes (14/7)
---------------------------------	-------------------	-------------------------------------

Sim, o plano de

refatoração sugerido resolvia corretamente a ocorrência do smell. Ele apresenta, como em todos, que não é bug funcional, mas é um alerta de manutenção. Ele recomenda agrupar a configuração de desenho em um objeto separado

Link da publicação OpenCode: <https://opncd.ai/share/TC7qMhGZ>

Print de consumo do token:



Análise da refatoração: Resolveu e ocorreu rápido. Mas, nesse caso, ele precisou refatorar dois arquivos

3.2.5 Ocorrência Número 5

Tabela de identificação

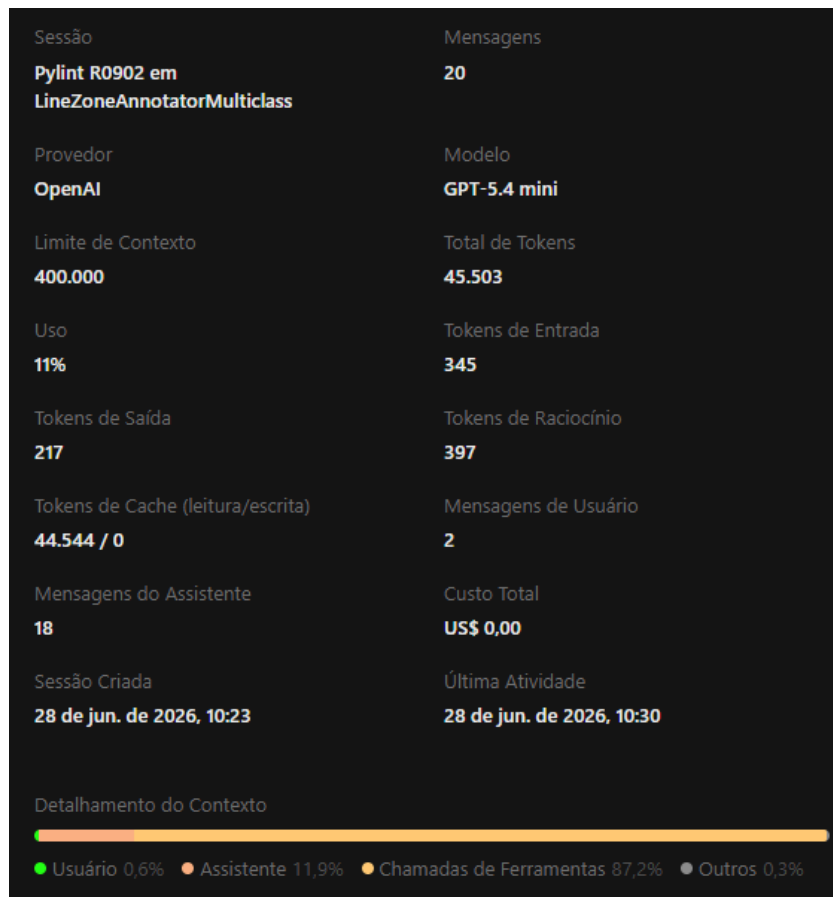
module	obj	message
supervision.detection.	LineZoneAnnotatorMulticlass	Too many instance attributes (9/7)

**Seguiu o plano
de refatoração:**

Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell. Novamente problema de design de sistema. Só uma classe acumulando responsabilidades demais

Link da publicação OpenCode: <https://opncd.ai/share/QffbmGBe>

Print de consumo do token:



Análise da refatoração: A análise apontou que a classe concentrava responsabilidades relacionadas à configuração e ao comportamento da anotação. A separação dessas responsabilidades reduziu o número de atributos e tornou a estrutura mais aderente aos princípios de projeto.

3.2.6 Ocorrência Número 6

Tabela de identificação

**Seguiu o plano
de refatoração:**

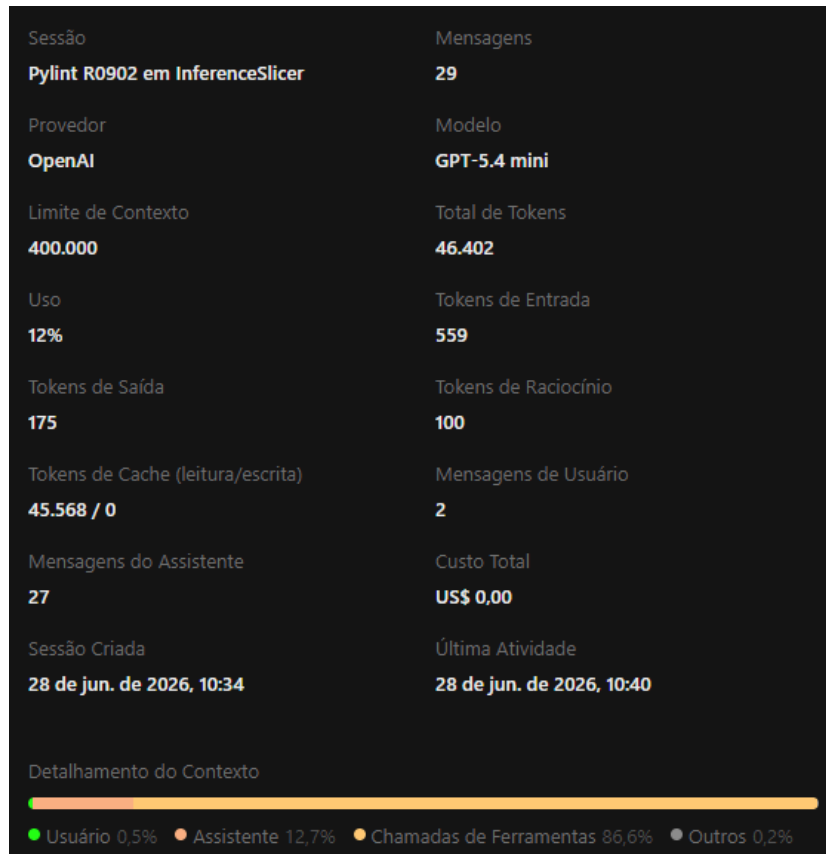
Sim, o plano de refatoração

module	obj	message
supervision.detection.tools.inference_slicer	InferenceSlicer	Too many instance attributes (12/7)

sugerido resolvia corretamente a ocorrência do smell. Eel basicamente reduziu o número de atributos da classe principal movendo flags/locks para um contêiner interno

Link da publicação OpenCode: <https://opncd.ai/share/QB2JZLLz>

Print de consumo do token:



Análise de refatoração: A refatoração agrupou atributos relacionados ao estado interno em uma estrutura auxiliar, reduzindo a quantidade de atributos expostos pela classe principal e tornando sua responsabilidade mais bem definida.

3.2.7 Ocorrência Número 7

Tabela de identificação

Seguiu o plano de refatoração:

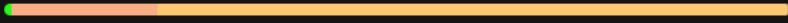
Sim, o plano de refatoração

module	obj	message
supervision.detection.tools.polygon_zone	PolygonZoneAnnotator	Too many instance attributes (11/7)

sugerido resolvia corretamente a ocorrência do smell. Ele apontou que a classe misturá papéis e recomendou uma separação por uma série de passos

Link da publicação OpenCode: <https://opncd.ai/share/92dCUOgb>

Print de consumo do token:

Sessão	Mensagens
Pylint R0902 no PolygonZoneAnnotator	17
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	29.113
Uso	Tokens de Entrada
7%	231
Tokens de Saída	Tokens de Raciocínio
206	516
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
28.160 / 0	2
Mensagens do Assistente	Custo Total
15	US\$ 0,00
Sessão Criada	Última Atividade
28 de jun. de 2026, 10:45	28 de jun. de 2026, 10:49
Detalhamento do Contexto	
	
● Usuário 0,9% ● Assistente 18,6% ● Chamadas de Ferramentas 80,1% ● Outros 0,4%	

Análise de refatoração: Essa aconteceu bem e rápido. Adicionou muitas linhas, mas por ser em um só arquivo aconteceu mais rápido

3.2.8 Ocorrência Número 8

Tabela de identificação

Seguiu o plano de refatoração:

Sim, o plano de refatoração

module	obj	message
supervision.metrics.f1_score	F1ScoreResult	Too many instance attributes (9/7)

sugerido resolvia corretamente a ocorrência do smell. Ele apontou que a classe misturá papéis e recomendou uma separação por uma série de passos

Link da publicação OpenCode: <https://opncl.ai/share/Wog73qkm>

Print de consumo do token:

Sessão	Mensagens
Pylint R0902 em F1ScoreResult	24
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	41.703
Uso	Tokens de Entrada
10%	99
Tokens de Saída	Tokens de Raciocínio
84	48
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
41.472 / 0	2
Mensagens do Assistente	Custo Total
22	US\$ 0,00
Sessão Criada	Última Atividade
28 de jun. de 2026, 10:52	28 de jun. de 2026, 10:58
Detalhamento do Contexto	
	
● Assistente 16,2% ● Chamadas de Ferramentas 81,8% ● Outros 2%	

Análise de refatoração: Demorou um pouco mais, mas acredito que foi só minha máquina. Ele fez a modificação de dois itens, o score e o um teste desse score

3.2.9 Ocorrência Número 9

Tabela de identificação

Seguiu o plano de refatoração:
Sim, o plano de refatoração

module	obj	message
supervision.metrics.mean_average_precision	MeanAveragePrecisionResult	Too many instance attributes (9/7)

sugerido resolvia corretamente a ocorrência do smell. Ele apontou um excesso em atributos conceituais, propondo extrair os resultados por tamanho em um contêiner separado

Link da publicação OpenCode: <https://opncd.ai/share/3OxzVCuY>

Print de consumo do token:

Sessão	Mensagens
Análise do erro R0902 no MeanAveragePrecisionResult	19
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	38.750
Uso	Tokens de Entrada
10%	631
Tokens de Saída	Tokens de Raciocínio
160	71
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
37.888 / 0	2
Mensagens do Assistente	Custo Total
17	US\$ 0,00
Sessão Criada	Última Atividade
28 de jun. de 2026, 12:09	28 de jun. de 2026, 12:14
Detalhamento do Contexto	
	
● Usuário 0,6% ● Assistente 19,2% ● Chamadas de Ferramentas 80% ● Outros 0,2%	

Análise de refatoração: Ele foi mais rápido que o comum, mesmo com a mudança de dois arquivos para correção do smell e a adição de pelo menos 100 linhas ao todo(somando os dois arquivos).

3.2.10 Ocorrência Número 10

Tabela de identificação

Seguiu o plano de refatoração:

Sim, o plano de refatoração

sugerido resolvia corretamente a ocorrência do smell. Ele apontou alta taxa de acoplamento(entrada, avaliação, caches e resultados). Apontou uma ótima solução: agrupar o estado interno em um objeto auxiliar único.

Link da publicação OpenCode: <https://opncd.ai/share/DeEbSExb>

Print de consumo do token:

module	obj	message
supervision.metrics.me an_average_precision	COCOEvaluator	Too many instance attributes (9/7)

Sessão	Mensagens
Análise do erro R0902 em COCOEvaluator	31
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	54.342
Uso	Tokens de Entrada
14%	317
Tokens de Saída	Tokens de Raciocínio
206	59
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
53.760 / 0	2
Mensagens do Assistente	Custo Total
29	US\$ 0,00
Sessão Criada	Última Atividade
28 de jun. de 2026, 12:17	28 de jun. de 2026, 12:23
Detalhamento do Contexto	
● Usuário 0,3% ● Assistente 11,4% ● Chamadas de Ferramentas 88% ● Outros 0,3%	

Análise de refatoração: Os testes passaram com tranquilidade e a refatoração ocorreu bem e rápido. Não sei se nesses casos em que o LLM já analisou algo do contexto afeta a forma e a velocidade em que o problema é resolvido

3.2.11 Ocorrência Número 11

Tabela de identificação

Seguiu o plano de refatoração:

Sim, o plano de refatoração

module	obj	message
supervision.metrics.me an_average_recall	MeanAverageRecallResult	Too many instance attributes (9/7)

sugerido resolvia corretamente a ocorrência do smell. O comum em todos os casos: alta taxa de acoplamento, misturando resultados com sub resultados opcionais.

Link da publicação OpenCode: <https://opncd.ai/share/IMtWcDMS>

Print de consumo do token:

Sessão	Mensagens
Pylint R0902 em mean_average_recall	26
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	57.439
Uso	Tokens de Entrada
14%	327
Tokens de Saída	Tokens de Raciocínio
217	63
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
56.832 / 0	2
Mensagens do Assistente	Custo Total
24	US\$ 0,00
Sessão Criada	Última Atividade
28 de jun. de 2026, 12:27	28 de jun. de 2026, 12:33
Detalhamento do Contexto	
● Usuário 0,3% ● Assistente 16,2% ● Chamadas de Ferramentas 83,2% ● Outros 0,3%	

Análise de refatoração: Durante a operação foram realizados muito ajustes devido a testes que a ferramenta ia realizando. demorou um pouco, mas nada absurdo

3.2.12 Ocorrência Número 12

Tabela de identificação

Seguiu o plano de refatoração:

Sim, o plano de refatoração

module	obj	message
supervision.metrics.precision	PrecisionResult	Too many instance attributes (9/7)

sugerido resolvia corretamente a ocorrência do smell. Ele apontou a função que está com esse code Smell, verificou como é construído esse resultado e a menor refatoração possível que funciona com o restante da API

Link da publicação OpenCode: <https://opncd.ai/share/Vc2QOBP6>

Print de consumo do token:

Sessão	Mensagens
Pylint R0902 em PrecisionResult	24
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	52.270
Uso	Tokens de Entrada
13%	361
Tokens de Saída	Tokens de Raciocínio
157	40
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
51.712 / 0	2
Mensagens do Assistente	Custo Total
22	US\$ 0,00
Sessão Criada	Última Atividade
28 de jun. de 2026, 12:37	28 de jun. de 2026, 12:42
Detalhamento do Contexto	

Análise de refatoração: Durante a operação foram realizados muito ajustes devido a testes que a ferramenta ia realizando. demorou um pouco, mas nada absurdo

3.2.13 Ocorrência Número 13

Tabela de identificação

Seguiu o plano de refatoração:

Sim, o plano de refatoração

sugerido resolvia corretamente a ocorrência do smell. Ele apontou a função que está com esse code Smell, verificou como é construído esse resultado e a menor refatoração possível que funciona com o restante da API

Link da publicação OpenCode: <https://opncd.ai/share/zKtmp64L>

Print de consumo do token:

module	obj	message
supervision.metrics.rec all	RecallResult	Too many instance attributes (9/7)

Sessão	Mensagens
Análise do erro R0902 em RecallResult	21
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	34.573
Uso	Tokens de Entrada
9%	1.545
Tokens de Saída	Tokens de Raciocínio
186	74
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
32.768 / 0	2
Mensagens do Assistente	Custo Total
19	US\$ 0,00
Sessão Criada	Última Atividade
28 de jun. de 2026, 12:46	28 de jun. de 2026, 12:50

Detalhamento do Contexto

● Usuário 0,7%
 ● Assistente 12,9%
 ● Chamadas de Ferramentas 86,3%
 ● Outros 0,1%

Análise de refatoração: Muito rápido. seguiu o plano como montou e funcionou

3.2.14 Ocorrência Número 14

Tabela de identificação

Seguiu o plano de refatoração:

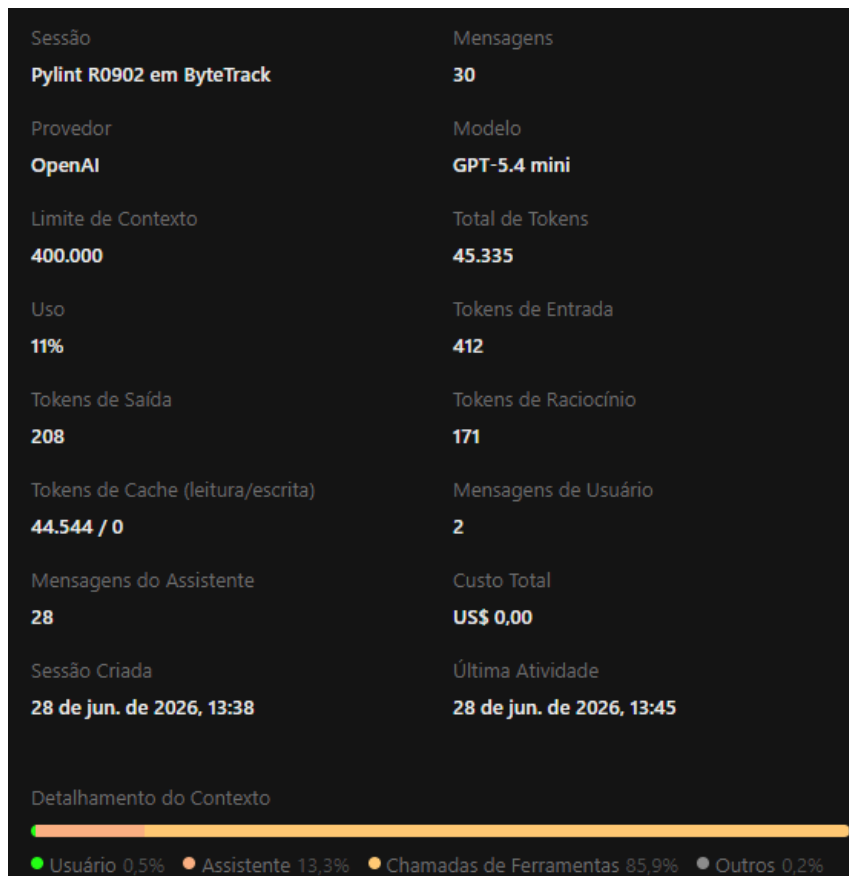
Sim. O plano de refatoração foi

module	obj	message
supervision.tracker.byte_tracker.core	ByteTrack	Too many instance attributes (13/7)

seguido, pois a LLM identificou corretamente que o excesso de atributos estava relacionado à concentração de configurações, estado de execução e dependências em uma única classe. A solução proposta consistiu em encapsular esses elementos em dois objetos auxiliares, reduzindo o acoplamento da classe principal sem alterar sua interface pública ou seu comportamento.

Link da publicação OpenCode: <https://opnecd.ai/share/iSo7o67A>

Print de consumo do token:



Análise de refatoração: Sim. O plano de refatoração foi seguido, pois a LLM identificou corretamente que o excesso de atributos estava relacionado à concentração de configurações, estado de execução e dependências em uma única classe. A solução proposta consistiu em encapsular esses elementos em dois objetos auxiliares, reduzindo o acoplamento da classe principal sem alterar sua interface pública ou seu comportamento.

3.2.15 Ocorrência Número 15

Tabela de identificação

Seguiu o plano de refatoração:

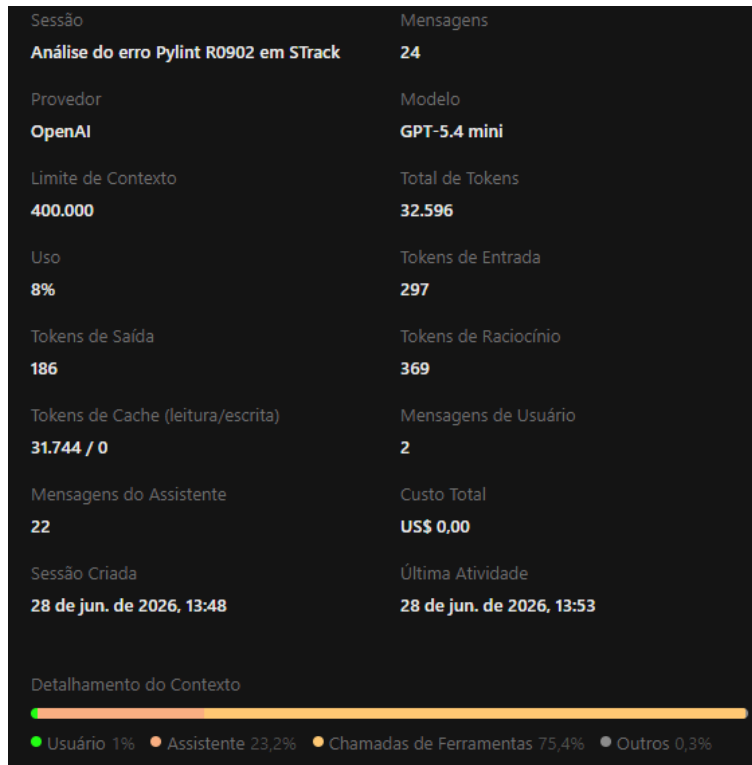
Sim. O plano de refatoração foi seguido, pois a

module	obj	message
supervision.tracker.byte_tracker.single_object_track	STrack	Too many instance attributes (16/7)

LLM identificou que a classe `STrack` concentrava diferentes responsabilidades, como gerenciamento do ciclo de vida do rastreamento, estado do filtro de Kalman, controle de identificadores e configurações operacionais. A solução adotada consistiu na remoção de um atributo redundante e na reorganização dessas responsabilidades em componentes mais coesos, preservando a interface pública e o comportamento esperado da classe.

Link da publicação OpenCode: <https://opncd.ai/share/qw4GksU0>

Print de consumo do token:



Análise de refatoração: A refatoração foi concluída com sucesso e solucionou o problema de excesso de atributos sem alterar o funcionamento do rastreador. Além da reorganização estrutural da classe, a LLM identificou um atributo que poderia ser removido por ser redundante, reduzindo ainda mais a complexidade da implementação. Após a conclusão das alterações, foi realizada a validação por meio dos testes específicos do módulo, confirmando que o comportamento da funcionalidade foi preservado. A estratégia adotada demonstrou uma boa compreensão da arquitetura do projeto e produziu uma solução consistente para o code smell apontado pelo Pylint.

3.3 too-many-branches

aqui foi usado o modelo da OpenAi GPT 5.1 Mini

3.3.1 Ocorrência Número 1

Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

module	obj	message
--------	-----	---------

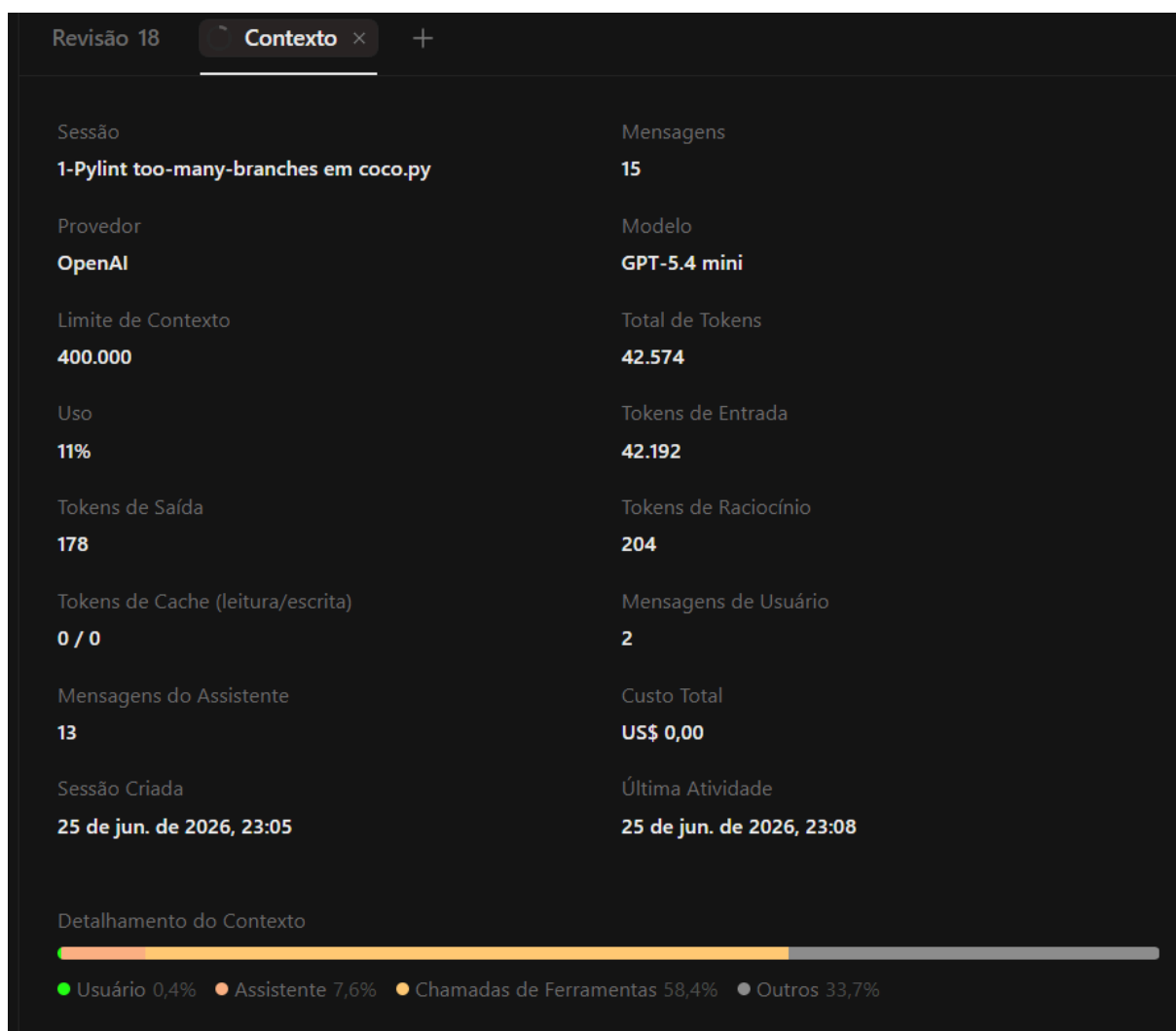
supervision.dataset.formats. coco	detections_to_coco_annotations	too many branches (14/12)
--------------------------------------	--------------------------------	---------------------------

Seguiu o plano

de refatoração: Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Link da publicação OpenCode: <https://opncl.ai/share/OfopY0zJ>

Print consumo de token:



Por fim, você deve fazer uma análise da refatoração dessa ocorrência. Por exemplo, avalie se valeu a pena utilizar a LLM para refatorar a ocorrência, considerando o consumo de tokens e o tempo gasto.

Análise da refatoração: A refatoração funcionou, removeu o smell e o tempo de refatoração foi rápido

3.3.2 Ocorrência Número 2

Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

Seguiu o plano de refatoração:

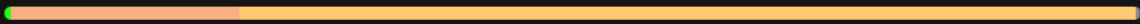
module	obj	message
supervision.detection.core	Detections.from_sam3	Too many branches (15/12)

Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Também deve informar o link da publicação gerada no OpenCode e adicionar um print do consumo de tokens referente à refatoração dessa ocorrência.

Link da publicação OpenCode: <https://opncd.ai/share/FqXdQRRp>

Print consumo de token:

Sessão	Mensagens
2 - upervision.detection.core	20
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	35.605
Uso	Tokens de Entrada
9%	504
Tokens de Saída	Tokens de Raciocínio
226	59
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
34.816 / 0	2
Mensagens do Assistente	Custo Total
18	US\$ 0,00
Sessão Criada	Última Atividade
25 de jun. de 2026, 23:09	25 de jun. de 2026, 23:13
Detalhamento do Contexto 	
● Usuário 0,6% ● Assistente 20% ● Chamadas de Ferramentas 79% ● Outros 0,4%	

Por fim, você deve fazer uma análise da refatoração dessa ocorrência. Por exemplo, avalie se valeu a pena utilizar a LLM para refatorar a ocorrência, considerando o consumo de tokens e o tempo gasto.

Análise da refatoração: A refatoração funcionou, removeu o smell e o tempo de refatoração foi rápido

3.3.3 Ocorrência Número 3

Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

Seguiu o plano de refatoração:

module	obj	message
supervision.detection.vlm	from_google_gemini_2_5	Too many branches (23/12)

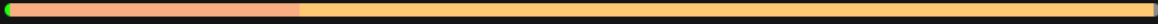
Sim, o plano de

refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Também deve informar o link da publicação gerada no OpenCode e adicionar um print do consumo de tokens referente à refatoração dessa ocorrência.

Link da publicação OpenCode: <https://opncd.ai/share/X8PR6s0o>

Print consumo de token:

Sessão	Mensagens
3 - supervision.detection.vlm	23
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	38.320
Uso	Tokens de Entrada
10%	215
Tokens de Saída	Tokens de Raciocínio
183	34
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
37.888 / 0	2
Mensagens do Assistente	Custo Total
21	US\$ 0,00
Sessão Criada	Última Atividade
25 de jun. de 2026, 23:14	25 de jun. de 2026, 23:19
Detalhamento do Contexto  ● Usuário 0,5% ● Assistente 25,1% ● Chamadas de Ferramentas 74% ● Outros 0,5%	

Por fim, você deve fazer uma análise da refatoração dessa ocorrência. Por exemplo, avalie se valeu a pena utilizar a LLM para refatorar a ocorrência, considerando o consumo de tokens e o tempo gasto.

Análise da refatoração: A refatoração funcionou, removeu o smell e o tempo de refatoração foi rápido

3.3.4 Ocorrência Número 4

Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

Seguiu o plano de refatoração:

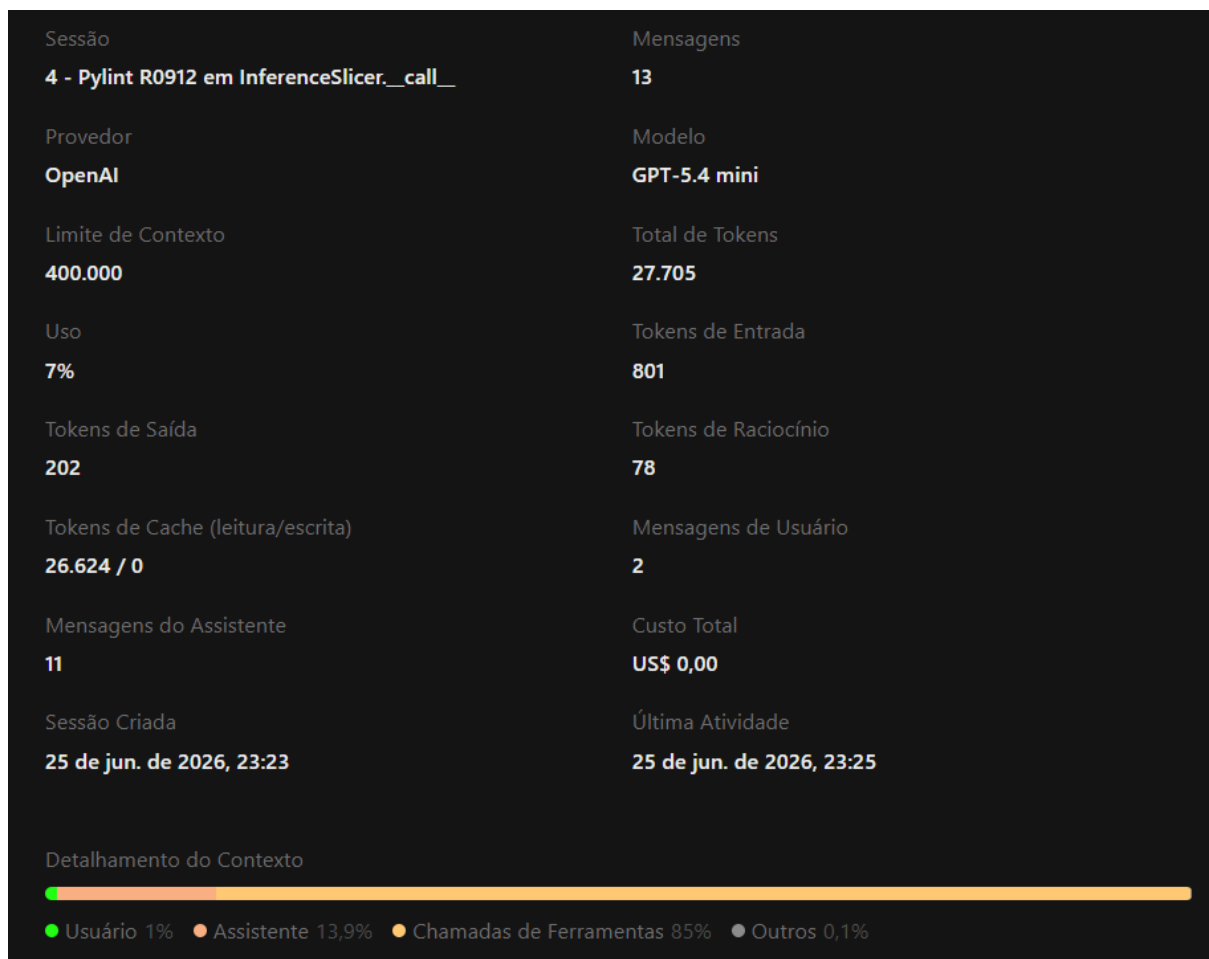
module	obj	message
supervision.detection.tools.inference_slicer	InferenceSlicer.__call__	oo many branches (13/12)

Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Também deve informar o link da publicação gerada no OpenCode e adicionar um print do consumo de tokens referente à refatoração dessa ocorrência.

Link da publicação OpenCode: <https://opncd.ai/share/ecwzvZKt>

Print consumo de token:



Por fim, você deve fazer uma análise da refatoração dessa ocorrência. Por exemplo, avalie se valeu a pena utilizar a LLM para refatorar a ocorrência, considerando o consumo de tokens e o tempo gasto.

Análise da refatoração: A refatoração funcionou, removeu o smell e o tempo de refatoração foi rápido

3.3.5 Ocorrência Número 5

Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

	module	obj	message
Seguiu o plano de refatoração:	supervision.detection. utils.internal	merge_data	Too many branches (13/12)

Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Também deve informar o link da publicação gerada no OpenCode e adicionar um print do consumo de tokens referente à refatoração dessa ocorrência.

Link da publicação OpenCode: <https://opncd.ai/share/i8eeWW3B>

Print consumo de token:

Sessão	Mensagens
5 - Pylint too-many-branches em merge_data	14
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	19.388
Uso	Tokens de Entrada
5%	207
Tokens de Saída	Tokens de Raciocínio
160	77
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
18.944 / 0	3
Mensagens do Assistente	Custo Total
11	US\$ 0,00
Sessão Criada	Última Atividade
25 de jun. de 2026, 23:28	25 de jun. de 2026, 23:29
Detalhamento do Contexto  ● Usuário 1,9% ● Assistente 24,6% ● Chamadas de Ferramentas 72,5% ● Outros 1%	

Por fim, você deve fazer uma análise da refatoração dessa ocorrência. Por exemplo, avalie se valeu a pena utilizar a LLM para refatorar a ocorrência, considerando o consumo de tokens e o tempo gasto.

Análise da refatoração: A refatoração funcionou, removeu o smell e o tempo de refatoração foi rápido

3.3.6 Ocorrência Número 5

Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

Seguiu o plano de refatoração:

Sim, o plano de

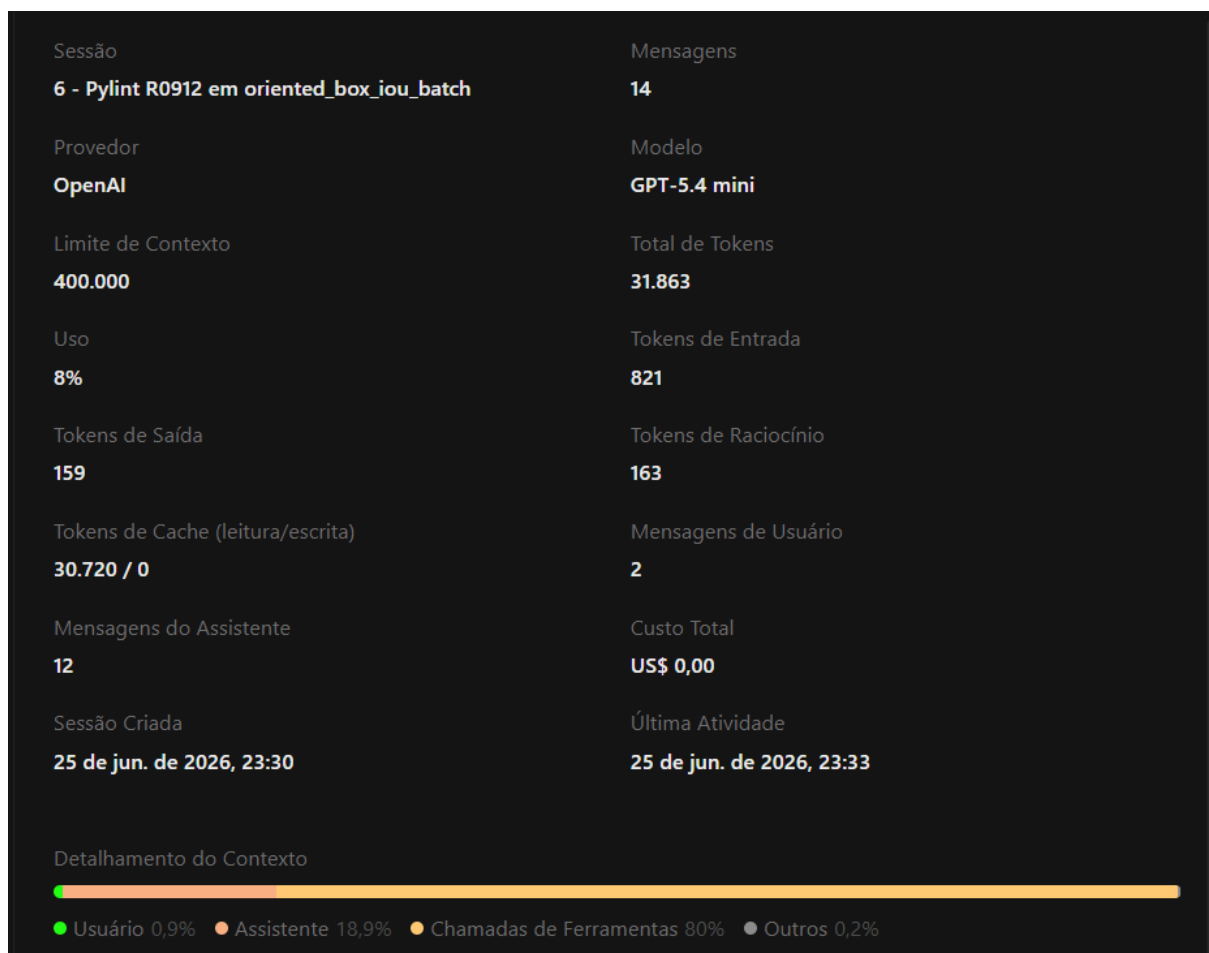
refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

module	obj	message
supervision.detection. utils.iou_and_nms	oriented_box_iou_batch	Too many branches (13/12)

Também deve informar o link da publicação gerada no OpenCode e adicionar um print do consumo de tokens referente à refatoração dessa ocorrência.

Link da publicação OpenCode: <https://opncd.ai/share/IY8NYV3f>

Print consumo de token:



Por fim, você deve fazer uma análise da refatoração dessa ocorrência. Por exemplo, avalie se valeu a pena utilizar a LLM para refatorar a ocorrência, considerando o consumo de tokens e o tempo gasto.

Análise da refatoração: A refatoração funcionou, removeu o smell e o tempo de refatoração foi rápido

3.3.7 Ocorrência Número 7

Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

	module	obj	message
Seguiu o plano de refatoração:	supervision.detection. utils.masks	filter_segments_by_distance	Too many branches (13/12)

Sim, o plano de

refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Também deve informar o link da publicação gerada no OpenCode e adicionar um print do consumo de tokens referente à refatoração dessa ocorrência.

Link da publicação OpenCode: <https://opncd.ai/share/ecV7FPgh>

Print consumo de token:

Sessão	Mensagens
7 - Pylint R0912 em filter_segments_by_distance	12
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	20.009
Uso	Tokens de Entrada
5%	2.265
Tokens de Saída	Tokens de Raciocínio
157	179
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
17.408 / 0	2
Mensagens do Assistente	Custo Total
10	US\$ 0,00
Sessão Criada	Última Atividade
25 de jun. de 2026, 23:40	25 de jun. de 2026, 23:42
Detalhamento do Contexto  ● Usuário 1,9% ● Assistente 24,4% ● Chamadas de Ferramentas 73,6% ● Outros 0%	

Por fim, você deve fazer uma análise da refatoração dessa ocorrência. Por exemplo, avalie se valeu a pena utilizar a LLM para refatorar a ocorrência, considerando o consumo de tokens e o tempo gasto.

Análise da refatoração: A refatoração funcionou, removeu o smell e o tempo de refatoração foi rápido

3.3.8 Ocorrência Número 8

Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

Seguiu o plano de refatoração:

Sim, o plano de

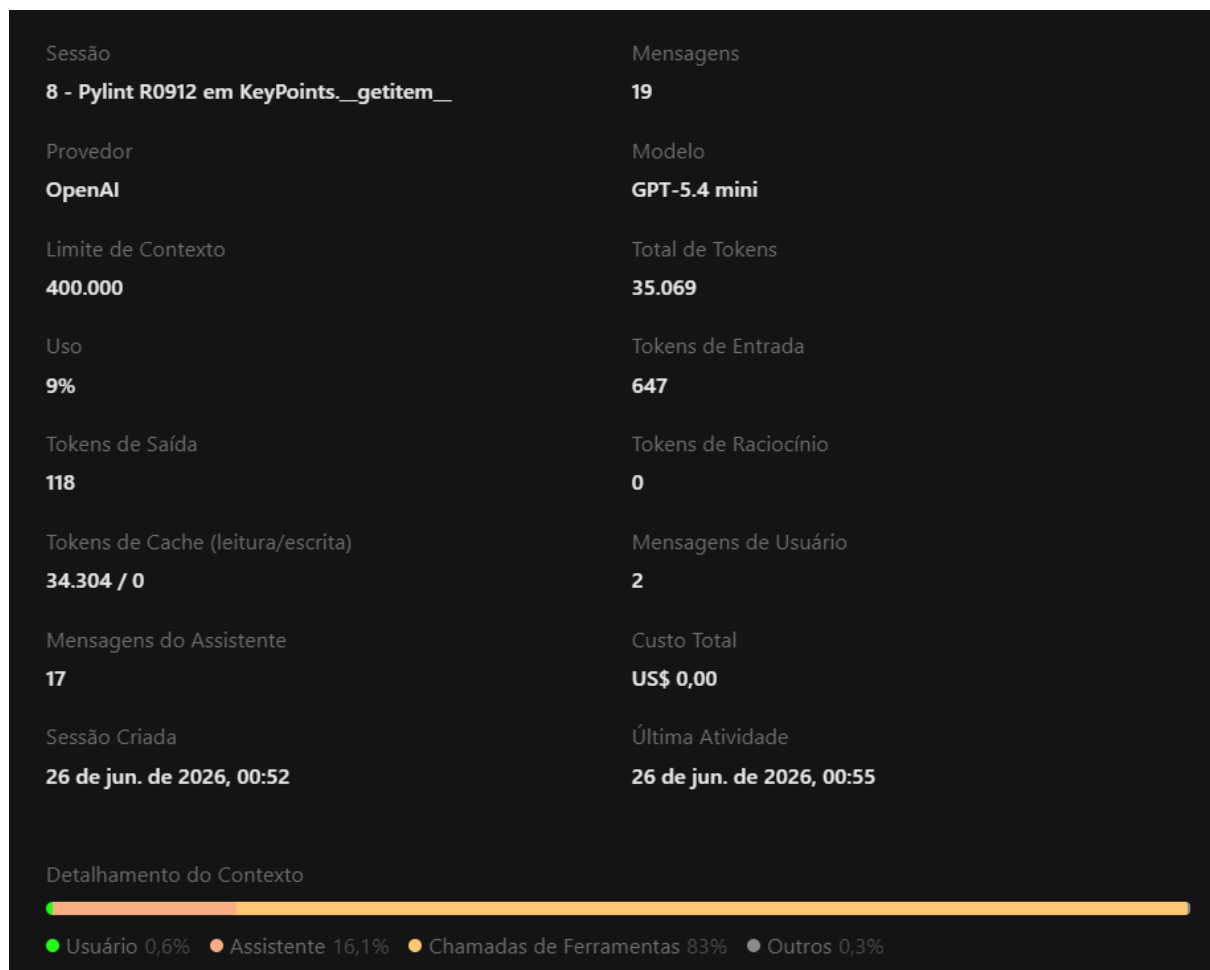
refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

module	obj	message
supervision.key_point s.core	KeyPoints._getitem__	Too many branches (22/12)

Também deve informar o link da publicação gerada no OpenCode e adicionar um print do consumo de tokens referente à refatoração dessa ocorrência.

Link da publicação OpenCode: <https://opncd.ai/share/XXkqlyF1>

Print consumo de token:



Por fim, você deve fazer uma análise da refatoração dessa ocorrência. Por exemplo, avalie se valeu a pena utilizar a LLM para refatorar a ocorrência, considerando o consumo de tokens e o tempo gasto.

Análise da refatoração: A refatoração funcionou, removeu o smell e o tempo de refatoração foi rápido

3.3.9 Ocorrência Número 9

Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

module	obj	message
supervision.metrics.detection	ConfusionMatrix.evaluate_detection_batch	Too many branches (16/12)

Seguiu o plano de refatoração:

Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Também deve informar o link da publicação gerada no OpenCode e adicionar um print do consumo de tokens referente à refatoração dessa ocorrência.

Link da publicação OpenCode: <https://opncd.ai/share/GIsZQjIq>

Print consumo de token:

Sessão	Mensagens
9 - Pylint too-many-branches em detection.py	18
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	34.358
Uso	Tokens de Entrada
9%	855
Tokens de Saída	Tokens de Raciocínio
142	81
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
33.280 / 0	2
Mensagens do Assistente	Custo Total
16	US\$ 0,00
Sessão Criada	Última Atividade
26 de jun. de 2026, 00:56	26 de jun. de 2026, 01:00
Detalhamento do Contexto  ● Usuário 0,7% ● Assistente 18,9% ● Chamadas de Ferramentas 80,2% ● Outros 0,1%	

Por fim, você deve fazer uma análise da refatoração dessa ocorrência. Por exemplo, avalie se valeu a pena utilizar a LLM para refatorar a ocorrência, considerando o consumo de tokens e o tempo gasto.

Análise da refatoração: A refatoração funcionou, removeu o smell e o tempo de refatoração foi rápido

3.3.10 Ocorrência Número 10

Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

Seguiu o plano de refatoração:

Sim, o plano de

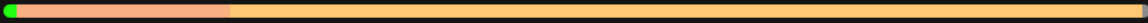
refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

module	obj	message
supervision.tracker.byte_tracker.core	ByteTrack.update_with_tensors	Too many branches (21/12)

Também deve informar o link da publicação gerada no OpenCode e adicionar um print do consumo de tokens referente à refatoração dessa ocorrência.

Link da publicação OpenCode: <https://opncd.ai/share/M4Fdx6lE>

Print consumo de token:

Sessão	Mensagens
10 - Pylint R0912 em ByteTrack.update_with_tensors	10
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	24.409
Uso	Tokens de Entrada
6%	167
Tokens de Saída	Tokens de Raciocínio
178	1.536
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
22.528 / 0	2
Mensagens do Assistente	Custo Total
8	US\$ 0,00
Sessão Criada	Última Atividade
26 de jun. de 2026, 18:31	26 de jun. de 2026, 18:34
Detalhamento do Contexto	
	
● Usuário 1,2% ● Assistente 18,6% ● Chamadas de Ferramentas 79,6% ● Outros 0,6%	

Por fim, você deve fazer uma análise da refatoração dessa ocorrência. Por exemplo, avalie se valeu a pena utilizar a LLM para refatorar a ocorrência, considerando o consumo de tokens e o tempo gasto.

Análise da refatoração: A refatoração funcionou, removeu o smell e o tempo de refatoração foi rápido

3.3.11 Ocorrência Número 11

Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

	module	obj	message
Seguiu o plano de refatoração:	supervision.utils.video	process_video	Too many branches (14/12)

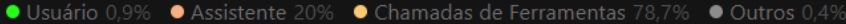
Sim, o plano de

refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Também deve informar o link da publicação gerada no OpenCode e adicionar um print do consumo de tokens referente à refatoração dessa ocorrência.

Link da publicação OpenCode: <https://opncd.ai/share/WSmXED3Z>

Print consumo de token:

Sessão	Mensagens
11 - Pylint too-many-branches em process_video	13
Provedor	Modelo
OpenAI	GPT-5.4 mini
Limite de Contexto	Total de Tokens
400.000	25.789
Uso	Tokens de Entrada
6%	465
Tokens de Saída	Tokens de Raciocínio
169	67
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
25.088 / 0	2
Mensagens do Assistente	Custo Total
11	US\$ 0,00
Sessão Criada	Última Atividade
26 de jun. de 2026, 18:36	26 de jun. de 2026, 18:38
Detalhamento do Contexto	
	
	

Por fim, você deve fazer uma análise da refatoração dessa ocorrência. Por exemplo, avalie se valeu a pena utilizar a LLM para refatorar a ocorrência, considerando o consumo de tokens e o tempo gasto.

Análise da refatoração: A refatoração funcionou, removeu o smell e o tempo de refatoração foi rápido

4. Métricas Após a Refatoração

Para as métricas após a refatoração você deve adicionar os mesmos tipos de informações que foram apresentadas anteriormente. Atenção, certifique-se de que cada um dos dados estão nas pastas metrics-after-...

4.1 Radon Após a Refatoração

4.1.1 Complexidade Ciclomática por Arquivo Após a Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela depois da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
src/supervision/key_points/core.py	22	6.14	39	135	E	getitem
src/supervision/detection/tools/smooth.py	4	6.50	9	26	B	get_track
src/supervision/detection/trackers/polygons.py	2	8.00	9	16	B	filter_polygons_by_area
src/supervision/detection/trackers/vlm.py	22	6.05	33	133	C	from_google_gemini_2_5
src/supervision/detection/trackers/internal.py	8	10.12	22	81	D	process_roboflow_result

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
src/supervision/annotators/base.py	1	1.00	1	1	A	annotate
src/supervision/geometry/core.py	11	1.00	1	11	A	list
src/supervision/metrics/core.py	3	1.00	1	3	A	update
src/supervision/tracker/byte_tracker/trackers.py	4	1.25	2	5	A	init
src/supervision/detection/tools/polygon_zone.py	22	1.27	4	28	A	annotate

4.1.2 Complexidade Ciclomática por Função Após a Refatoração

Apresente as funções do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela depois da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
src/supervision/tracker/byte_tracker/core.py	method	update_with_tensors	D	26
src/supervision/metrics/mean_average_precision.py	method	evaluate_image	D	28
src/supervision/metrics/mean_average_precision.py	method	accumulate	D	29
src/supervision/detection/vlm.py	function	from_google_gemini_2_5	E	33
src/supervision/key_points/core.py	method	getitem	E	39

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
src/supervision/annotators/base.py	method	annotate	A	1
src/supervision/annotators/core.py	function	normalize_color_input	A	1
src/supervision/annotators/core.py	method	init	A	1
src/supervision/annotators/core.py	method	color	A	1
src/supervision/annotators/core.py	method	color_lookup	A	1

4.1.3 Índice de Manutenibilidade Após a Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela depois da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	mi	rank_mi
src/supervision/annotators/core.py	0.0000	C
src/supervision/metrics/mean_average_precision.py	5.4662	C
src/supervision/detection/core.py	15.2378	B
src/supervision/detection/compact_mask.py	25.2684	A
src/supervision/detection/utils/converters.py	26.8962	A

Tabela de melhores métricas

arquivo	mi	rank_mi
src/supervision/metrics/init.py	100.0000	A
src/supervision/metrics/utils/utils.py	100.0000	A
src/supervision/metrics/utils/init.py	100.0000	A
src/supervision/tracker/init.py	100.0000	A
src/supervision/tracker/byte_tracker/init.py	100.0000	A

4.1.4 Métricas Brutas Após a Refatoração

Essas métricas podem ser encontradas no arquivo *raw_por_arquivo_e_total_depois.csv* Traga apenas o total, que se encontra no final do csv.

loc total	lloc total	sloc total	comments total	multi
34876	11386	20295	516	9110

4.2 CodeCarbon Após a Refatoração

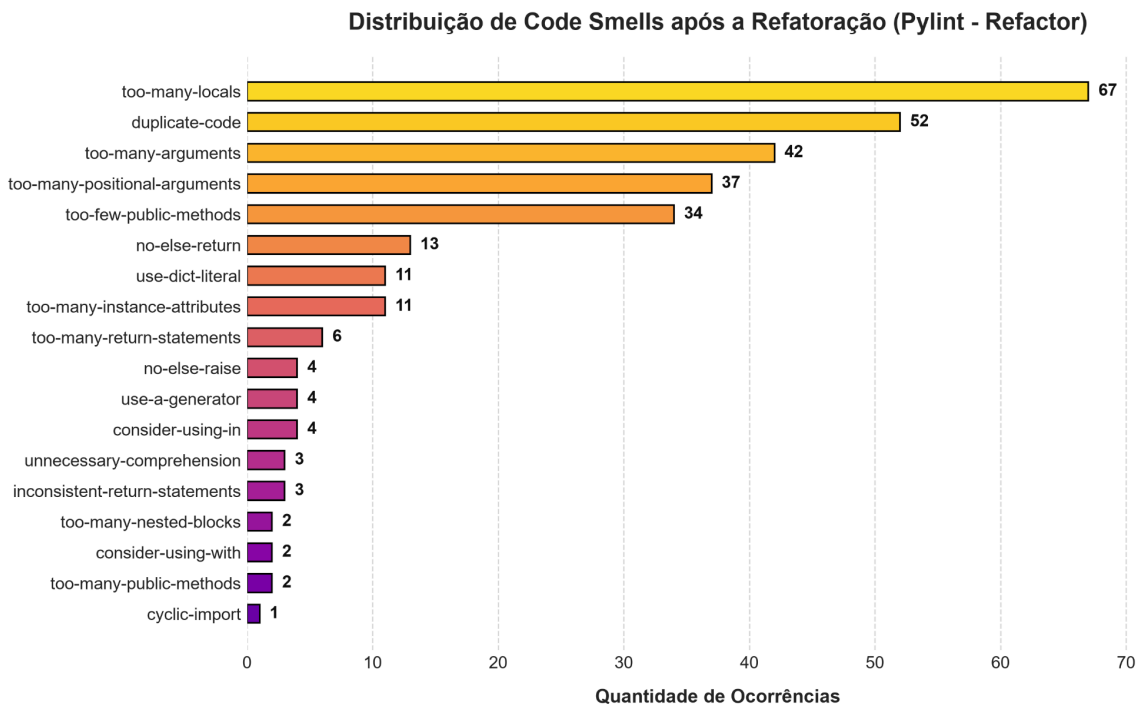
Preencha a tabela abaixo com as métricas ambientais depois da refatoração.

duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
22.93 56486	0.00005177440 3370802725	0.0000022573 769015109355	0.00001687706 215654022	0.00003560395 855556	0.000526440836 3241013

4.3 Pylint Após a Refatoração

4.3.1 Distribuição do Smells Após a Refatoração

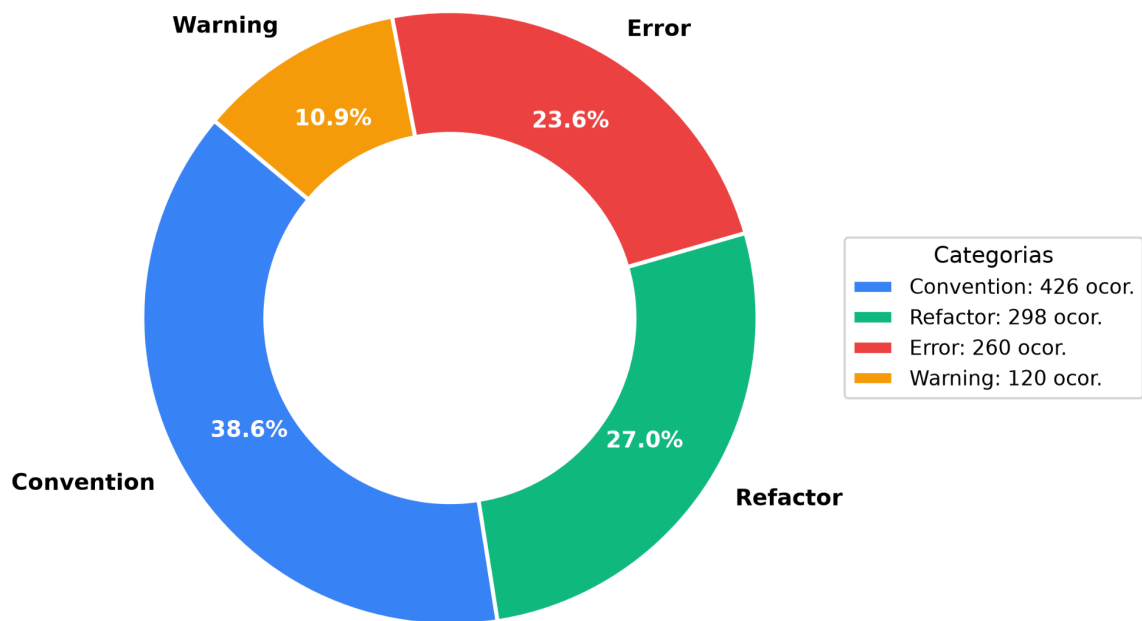
A partir do JSON `pylint_ranking_smells_depois.json` gere um gráfico para representar a distribuição dos code smells depois da refatoração. O gráfico precisa ter um bom contraste para facilitar a visualização.



4.3.2 Distribuição dos Problemas de Código Após a Refatoração

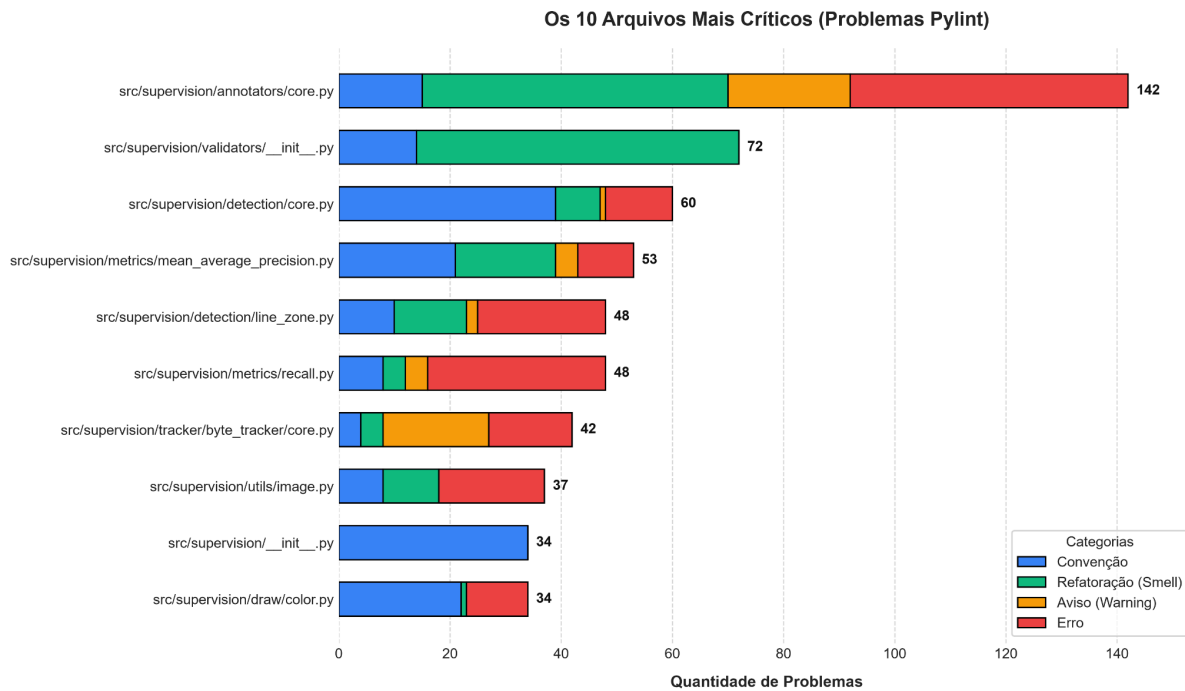
A partir do JSON `pylint_distribicao_categorias_depois.json` gere um gráfico para representar a distribuição entre as 4 categorias de problemas de código depois da refatoração. O gráfico precisa ter um bom contraste para facilitar a visualização.

Distribuição das Categorias de Problemas de Código (Pylint)



4.3.3 Arquivos mais Críticos Após a Refatoração

A partir do JSON `pylint_arquivos_criticos_depois.json` gere um gráfico para representar a distribuição dos 10 arquivos mais críticos depois da refatoração. O gráfico precisa ter um bom contraste para facilitar a visualização.



4.3.4 Score Projeto Após a Refatoração

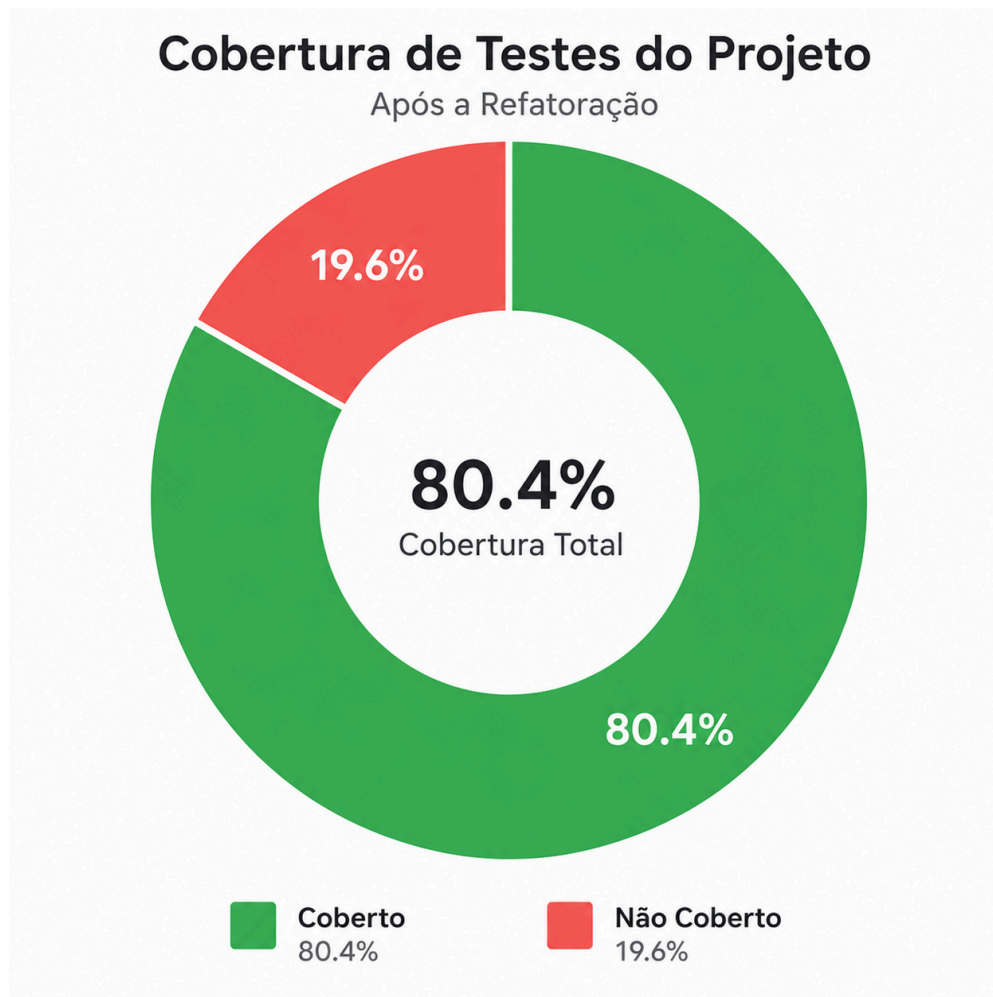
A partir do text *pylint_score_depois.txt* informe o score que o pylint deu ao projeto depois da refatoração. Não precisa ser uma imagem, fica a critério de vocês.

Your code has been rated at 7.62/10 (previous run: 7.73/10, +0.00)

4.4 Pytest Após a Refatoração

4.4.1 Cobertura de testes Após a Refatoração

Gere um gráfico da cobertura de testes do projeto depois da refatoração. Para isso, basta visualizar a cobertura de testes na página HTML, copiar os dados da tabela e gerar o gráfico com o auxílio de uma LLM. Para visualizar a página de cobertura de testes depois da refatoração, execute o seguinte comando: `start "metrics-after-pytest\coverage_depois_html\index.html"`. O gráfico precisa ter um bom contraste para facilitar a visualização.



4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração

Aqui você deve informar quantos testes falharam e quantos testes passaram depois da refatoração. Para isso, basta visualizar a página HTML `pytest_depois.html`. Para acessá-la, execute o seguinte comando: `start "metrics-after-pytest\pytest_depois.html"`

Quantidade de testes que passaram: 2031 Passed

Quantidade de testes que falharam: 44 Failed

5. Comparação dos Resultados

Após a refatoração, o projeto apresentou uma melhora localizada na organização de alguns módulos, mas também deixou claro que ainda existem pontos críticos concentrando complexidade e problemas de qualidade. No Radon, vários arquivos ficaram com complexidade ciclomática baixa e rank A, como `annotators/base.py`, `geometry/core.py`, `metrics/core.py` e `tracker/byte_tracker/utils.py`, o que indica partes do código mais simples e fáceis de manter. Também houve boa manutenção em arquivos de infraestrutura, com vários módulos tendo MI de 100.0, especialmente em `__init__.py` e utilitários menores. Por outro lado, persistem hotspots relevantes em arquivos centrais, como

`annotators/core.py`, `detection/core.py`, `metrics/mean_average_precision.py` e `key_points/core.py`, que continuam concentrando alta complexidade e piorando a legibilidade do projeto como um todo.

Na comparação com os resultados anteriores, o `pylint` piorou. O score caiu de 7.98/10 para 7.62/10, o que mostra que a refatoração não reduziu os problemas apontados pela ferramenta. A distribuição de categorias também ficou mais pesada: `convention` aumentou de 373 para 628, `error` de 202 para 233 e `warning` de 101 para 110, enquanto `refactor` ficou praticamente estável (303 para 307). Isso indica que, embora parte da estrutura do código tenha sido reorganizada, o projeto passou a acumular mais violações de estilo e mais problemas efetivos de código.

Nos testes, o cenário também piorou. Antes da refatoração, havia 2068 testes com apenas 4 falhas; depois, o relatório mostrou 2031 testes aprovados e 44 falhas. A cobertura geral ficou praticamente estável, mas com leve queda: o total exibido passou de 81% para 80%, enquanto `statements` e `branches` permaneceram em 84% e 65%. Isso sugere que a base de testes continua razoável, mas a refatoração introduziu regressões funcionais em áreas específicas, principalmente em `geometry` e `line_counter`, onde aparecem os erros relacionados ao uso de `np.cross`.

Em relação ao `CodeCarbon`, a execução após a refatoração registrou consumo baixo em termos absolutos, com 0.00005177 de emissões e 0.00052644 de energia consumida. Porém, a comparação direta com o antes não é muito fiel porque os artefatos anteriores mostram execuções diferentes, com tempos bem distintos. Ainda assim, a leitura geral é que o impacto ambiental da execução continua pequeno, sem indicar uma piora relevante nesse aspecto.

De forma geral, a refatoração trouxe ganhos pontuais na simplicidade de vários módulos, mas não consolidou uma melhora global na qualidade do projeto. Os principais ganhos ficaram na organização de partes menores e na manutenção de vários arquivos com baixa complexidade. Já as perdas mais claras apareceram no aumento de problemas apontados pelo `pylint` e no crescimento do número de testes falhos, o que mostra que a qualidade estrutural melhorou em alguns pontos, mas a qualidade funcional e a consistência do código ficaram piores em outros.

6. Percepções sobre a Prática

Experiência geral com a atividade:

No geral, a prática foi interessante porque permitiu observar o projeto por um ângulo mais analítico, indo além da leitura superficial do código e entrando em métricas reais de qualidade. Eu, particularmente, não me senti muito animado no começo por não ser uma área de interesse direta para mim, então a motivação inicial foi mais baixa do que em outras atividades. Mesmo assim, fazer algo diferente acabou sendo positivo, porque trouxe uma visão mais concreta sobre manutenção de software, qualidade interna do código e impacto de refatorações em um projeto grande. Também foi útil perceber que nem toda melhoria aparece de forma imediata no funcionamento do sistema, muitas vezes ela aparece em métricas, organização e facilidade de evolução. Isso deixou a atividade mais interessante do que eu imaginava no início, porque mostrou na prática como um projeto open source depende de consistência e cuidado contínuo.

Code smells:

Considero importante refatorar os code smells porque eles afetam diretamente a sustentabilidade de um projeto open source. Smells como too-many-locals, duplicate-code e too-many-arguments dificultam a leitura, aumentam a chance de erro e tornam mudanças futuras mais caras. Muitos locais em uma mesma função costumam indicar excesso de responsabilidade, o que prejudica a clareza e dificulta testes. Código duplicado aumenta a manutenção, porque qualquer alteração precisa ser replicada em vários pontos, elevando o risco de inconsistência. Já funções com muitos argumentos tendem a ser mais difíceis de entender, de chamar corretamente e de evoluir sem quebrar compatibilidade. Além desses, eu também considero importantes smells como too-many-branches, too-many-statements, too-many-instance-attributes e too-few-public-methods, porque todos eles apontam para estruturas mais frágeis e menos modulares. Como benefício, a refatoração melhora a legibilidade e deixa partes do projeto mais organizadas. Como limitação, nem sempre ela reduz a complexidade global, porque algumas áreas do domínio naturalmente exigem lógica extensa e continuam concentrando muitos problemas.

Uso da LLM na refatoração:

A LLM ajudou bastante a interpretar os problemas apontados pelo Pylint e a pensar em formas de reorganizar o código. Em muitos casos, as sugestões foram úteis e aplicáveis, principalmente quando o problema era evidente, como redução de duplicação, extração de funções ou simplificação de blocos condicionais. Ao mesmo tempo, nem toda sugestão servia diretamente para o contexto do projeto, então foi necessário revisar com cuidado para evitar mudanças excessivas ou soluções genéricas demais. No balanço geral, o uso da LLM acelerou o processo, especialmente na fase de análise e geração de alternativas, mas também exigiu validação humana para não introduzir regressões. Acredito que projetos open source podem se beneficiar muito desse tipo de apoio, desde que a LLM seja usada como ferramenta de assistência e não como substituta do julgamento técnico. Além da refatoração, ela também ajudou a estruturar a leitura das métricas, organizar comparações e transformar resultados técnicos em texto mais claro para documentação.

Contribuição com projeto open source:

Contribuir com um projeto open source foi um processo bom, mas com desafios reais. A principal dificuldade foi lidar com um código já grande e com muitas partes interdependentes, o que exige atenção para não mexer em um ponto e quebrar outro. Também foi importante entender o estilo do projeto e respeitar a arquitetura já existente, porque em open source não se trabalha só para “fazer funcionar”, mas para manter coerência com o restante da base. Ao mesmo tempo, o processo trouxe aprendizado sobre revisão de código, interpretação de métricas e responsabilidade com mudanças que afetam outros usuários. Eu acho que o que mais facilitaria contribuições desse tipo seria ter mais documentação objetiva, testes mais estáveis, exemplos mais claros e issues bem descritas. Isso reduz a curva de entrada e deixa a contribuição mais segura, especialmente para quem está começando em projetos maiores.

7. Links

- Link do fork https://github.com/EdivarCr/supervision_refactory
- Link do projeto <https://github.com/roboflow/supervision>
- Link do PR 1
https://github.com/EdivarCr/supervision_refactory/tree/refactor/remove-code-smell-nome
- Link do PR 2
https://github.com/EdivarCr/supervision_refactory/tree/refactor/remove-code-smell-instance-attributes
- link do PR de contribuição
<https://github.com/roboflow/supervision/pull/2377>